



MODUL PEMAHAMAN PROYEK

Memahami HOMCUTS

Alur kerja, arsitektur, dan kode aplikasi Laravel untuk pembaca yang memahami PHP dasar, MVC, dan OOP

BACKEND	FRONTEND	INFRASTRUKTUR
Laravel 13 PHP 8.4	Blade + Tailwind Vite + JavaScript	Docker Compose MySQL 8.4

Tujuan buku ini

Setelah membaca modul ini, Anda diharapkan mampu menjelaskan apa yang terjadi sejak pengguna menekan tombol sampai data tersimpan, memahami letak kode yang mengaturnya, menjalankan proyek dengan Docker, membaca struktur database, dan melakukan perubahan kecil secara aman.

Disusun berdasarkan source code dan konfigurasi proyek HOMCUTS
Pemeriksaan source: 28 September 2026

Cara menggunakan modul

Modul ini bukan dokumentasi Laravel umum. Seluruh contoh, nama file, URL, tabel database, dan alur bisnis diambil dari proyek HOMCUTS yang berada pada folder `da-project/barbershop-laravel`. Anda dapat membaca dari awal secara berurutan atau langsung menuju bab yang sedang diperlukan.

Karena Anda memahami PHP dasar, MVC, dan OOP, konsep Laravel dijelaskan dengan menghubungkannya ke class, object, method, pewarisan, dan pembagian tanggung jawab. Istilah database dan SQL yang dibutuhkan juga diterangkan saat pertama kali muncul.

Jika Anda ingin...	Baca bagian...
Memahami gambaran besar proyek	Bab 1–3
Menjelaskan setiap halaman kepada dosen/penguji	Bab 4–5
Memahami booking, produk, POS, pembayaran, dan antrean	Bab 6–8
Belajar membaca Laravel dari source code	Bab 9–11
Menjalankan, menguji, atau memperbaiki proyek	Bab 12–14
Mengetahui fungsi setiap file	Lampiran A–C

Daftar isi

Bab 1 · Gambaran besar HOMCUTS	Bab 2 · Laravel dari sudut pandang PHP dan MVC
Bab 3 · Arsitektur Docker dan lingkungan berjalan	Bab 4 · Tur halaman pelanggan
Bab 5 · Tur panel admin dan kasir	Bab 6 · Workflow booking secara mendalam
Bab 7 · Workflow toko produk	Bab 8 · POS, antrean, transaksi, dan pembayaran
Bab 9 · Notifikasi dan pembaruan otomatis	Bab 10 · Database dan model data
Bab 11 · Membaca struktur kode	Bab 12 · Frontend: Blade, Tailwind, dan JavaScript
Bab 13 · Keamanan dan konsistensi data	Bab 14 · Pengujian dan cara membaca test
Bab 15 · Panduan perubahan dan diagnosis	Lampiran A · Katalog route
Lampiran B · Inventaris folder dan file	Lampiran C · Indeks class dan fungsi
Lampiran D · Glosarium dan perintah cepat	

BAB 1

Gambaran besar HOMCUTS

Mulai dari masalah bisnis, aktor, dan batas tanggung jawab aplikasi sebelum melihat kode.

1.1 Apa yang dibangun

HOMCUTS adalah aplikasi web manajemen barbershop yang menyatukan situs pelanggan dan panel operasional admin/kasir. Pelanggan dapat melihat informasi usaha, memilih layanan dan capster, membuat booking, membeli produk untuk diambil di toko, melihat galeri, serta mengirim pesan. Admin mengelola data utama, booking, transaksi, pembayaran tunai, antrean, stok, galeri, dan informasi situs.

Aplikasi ini bersifat server-rendered: sebagian besar HTML dibuat oleh Laravel melalui Blade lalu dikirim ke browser. JavaScript dipakai untuk interaksi yang perlu terasa langsung—keranjang, pemeriksaan slot, polling status, penyortiran tabel tanpa reload penuh, modal, dan notifikasi. Tidak ada React, TypeScript, Inertia, Livewire, atau Vinext.

1.2 Aktor dan tanggung jawab

Aktor	Apa yang dilakukan	Batas akses
Pelanggan	Membaca informasi, booking, memesan produk, melihat status transaksi, mengirim pesan.	Tidak mempunyai akun dan tidak dapat membuka panel admin.
Admin/Kasir	Login, membuka POS, mengonfirmasi pembayaran tunai, mengubah jadwal, mengelola data, membaca laporan.	Harus login dan memiliki <code>is_admin = true</code> .
Scheduler	Memeriksa pembayaran yang melewati tenggat, membatalkan transaksi, melepas stok atau slot.	Berjalan otomatis di container terpisah; tidak mempunyai tampilan.
MySQL	Menyimpan data permanen dan menjaga relasi/transaksi.	Diakses oleh aplikasi, bukan langsung oleh pelanggan.

1.3 Prinsip bisnis penting

- Semua pembayaran aktif memakai tunai. QRIS/Midtrans tidak terhubung ke route publik saat ini.
- Booking dan walk-in memakai jadwal capster yang sama, sehingga keduanya saling memblokir bila waktunya bertabrakan.
- Nomor antrean layanan berasal dari satu urutan global (A001, A002, dan seterusnya), baik booking maupun POS.
- Pesanan produk tidak memakai pengiriman. Barang dibayar tunai dan diambil di barbershop.
- Status proses dan status pembayaran dipisahkan karena “sudah dibayar” belum selalu berarti “sudah selesai/diambil”.
- Transaksi yang dibatalkan tetap ada sebagai jejak audit pada riwayat lengkap, tetapi tidak tampil pada daftar transaksi terbaru dashboard.

Model mental satu kalimat

HOMCUTS adalah satu aplikasi Laravel yang menerima request dari browser, menjalankan aturan bisnis melalui controller dan service, menyimpan data melalui model Eloquent ke MySQL, lalu mengembalikan Blade/HTML atau JSON.

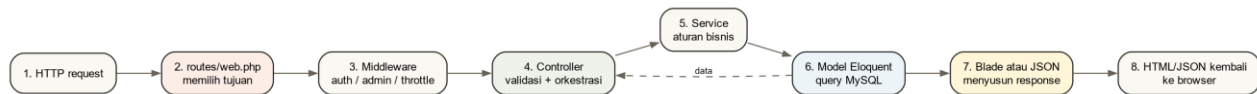
BAB 2

Laravel dari sudut pandang PHP dan MVC

Istilah Laravel baru akan lebih mudah dipahami jika dihubungkan ke PHP dasar, MVC, dan OOP yang sudah Anda kenal.

2.1 Padanan konsep

Konsep proyek	Padanan yang sudah dikenal	Peran pada HOMCUTS
Route	Pemilih file/controller berdasarkan URL	Menentukan method controller dan middleware untuk request.
Controller	Controller MVC	Menerima input, memvalidasi, memanggil service/model, memilih response.
Service	Kelas business logic	Menyimpan aturan kompleks agar controller tidak terlalu besar.
Model Eloquent	Model + query builder/DAO	Mewakili tabel dan relasi; menghasilkan query SQL.
Blade	Template PHP	Menggabungkan HTML dengan data server secara aman dan terstruktur.
Migration	DDL SQL berversi	Mencatat CREATE/ALTER/DROP agar struktur DB konsisten.
Seeder	INSERT data awal	Mengisi katalog, pengaturan, dan akun admin awal.
Middleware	Filter sebelum controller	Memeriksa login, admin, throttle, dan CSRF.
Vite	Build tool aset	Menggabungkan/mengoptimalkan CSS dan JS untuk browser.
Tailwind CSS	Kumpulan utility CSS	Gaya ditulis sebagai class pada Blade; Vite membangun CSS final.



Gambar 2.1 Siklus request pada aplikasi HOMCUTS

2.2 Contoh satu request sederhana

Ketika pelanggan membuka /gallery, browser mengirim GET. routes/web.php mengarahkan request ke PageController::gallery(). Controller mengambil galeri berstatus published beserta relasi capster. Data itu diberikan ke resources/views/gallery.blade.php. Blade menghasilkan HTML dan layout utama menambahkan navigasi, footer, CSS, serta JavaScript.

Untuk operasi perubahan data, alurnya menambah validasi dan transaksi database. Contohnya POST /bookings: controller memvalidasi formulir, service mengecek slot dan mengunci baris yang relevan, model menyimpan booking/order/payment, lalu response mengarahkan pengguna ke halaman status pembayaran.

Contoh hubungan route → controller → view

```

Route::get('/gallery', [PageController::class, 'gallery'])->name('gallery');

public function gallery(): View
{
    $entries = GalleryEntry::with('barber')
        ->where('is_published', true)
        ->orderBy('sort_order')
        ->get();
  
```

```
return view('gallery', compact('entries'));
}
```

Urutan lengkap dari request sampai response

Bayangkan pengguna memilih jadwal booking lalu menekan tombol Kirim. Browser tidak memanggil class Laravel secara langsung. Browser mengirim HTTP request ke web server; Laravel kemudian menentukan route, menjalankan kode PHP, dan mengirim HTTP response. Setiap langkah di bawah adalah bagian dari satu perjalanan itu.

1. **Browser membuat request.** Request berisi method seperti GET atau POST, path seperti /bookings, query string, header, cookie session, dan bila form dikirim, body berisi nilai input serta token CSRF. Pemeriksaan slot dari JavaScript memakai GET dan Accept: application/json; pengiriman booking memakai POST.
2. **Docker meneruskan koneksi.** Pada penggunaan lokal, port host 8000 diteruskan ke port 8000 pada container web. Server PHP di container menerima koneksi. Browser tidak berbicara langsung dengan MySQL; Laravel yang mengakses service db melalui jaringan Compose.
3. **Front controller menerima request.** Semua request web masuk melalui public/index.php. File ini memeriksa mode maintenance, memuat Composer autoloader, membuat aplikasi dari bootstrap/app.php, menangkap request sebagai object Illuminate\Http\Request, lalu menyerahkannya ke aplikasi untuk ditangani.
4. **Bootstrap menyiapkan aplikasi.** bootstrap/app.php mendaftarkan routes/web.php, routes/console.php, alias middleware admin, dan aturan exception. Provider mendaftarkan layanan aplikasi. Route definitions menjadi peta yang dapat dicocokkan router dengan URL dan method HTTP.
5. **Router memilih route.** Router mencocokkan method dan path request. GET /gallery menuju PageController::gallery; POST /bookings menuju BookingController::store. Nama route seperti bookings.store membantu membuat URL tanpa menulis path berulang kali.
6. **Middleware menyaring request.** Route web memakai middleware untuk cookie, session, dan CSRF. Route tertentu menambahkan throttle; route admin menambahkan auth dan admin. Middleware dapat meneruskan request, mengalihkan pengguna ke login, atau menghentikan request dengan status seperti 403, 419, atau 429.
7. **Container membangun controller.** Laravel membuat controller dan memasukkan dependency yang diminta di constructor. BookingController menerima Request, BookingAvailabilityService, BookingTransactionService, PaymentService, dan AdminNotifier. Untuk /payments/{payment}, route model binding mencari Payment memakai reference karena model Payment menentukan getRouteKeyName().
8. **Controller memvalidasi input.** Controller mengambil nilai dari Request dan memeriksa aturan seperti required, format tanggal, nilai yang diperbolehkan, serta keberadaan service aktif. Jika validasi gagal pada form web, Laravel mengembalikan pengguna ke halaman sebelumnya dengan pesan error dan old input di session. Jika request mengharapkan JSON, error dapat dikirim sebagai JSON dengan status 422.
9. **Aturan bisnis dijalankan.** Controller memanggil service untuk aturan yang lebih besar. BookingAvailabilityService menghitung durasi, jam buka, jam kerja capster, dan konflik jadwal. Untuk menyimpan booking, DB::transaction membungkus perubahan; lockForUpdate melindungi data penting dari dua request bersamaan.
10. **Eloquent berbicara dengan MySQL.** Model seperti Booking, Order, OrderItem, Product, dan Payment mewakili tabel. Query Eloquent diterjemahkan menjadi SQL. Perubahan dalam transaksi disimpan bersama-sama; bila terjadi exception sebelum transaksi selesai, MySQL membatalkan perubahan dalam transaksi tersebut.
11. **Controller memilih jenis response.** Controller dapat mengembalikan view, redirect, JSON, atau file PDF. Laravel mengubah hasil controller menjadi HTTP response lengkap dengan status code, header, cookie, session, dan body. Exception yang tidak ditangani diproses oleh exception handler.
12. **Blade membentuk HTML.** Untuk view, Blade menerima data controller dan layout. View composer di AppServiceProvider membagikan pengaturan situs serta katalog produk kepada view. Sintaks {{ ... }} meng-escape teks. Blade dirender di server; browser menerima HTML biasa, bukan file Blade.
13. **Browser memuat aset lalu melanjutkan interaksi.** Direktif @vite membaca manifest public/build dan memasukkan alamat CSS/JavaScript hasil build ke HTML. Browser meminta aset tersebut secara terpisah. Setelah halaman tampil, app.js dapat mengirim fetch baru untuk cek slot atau status; setiap fetch memulai request HTTP baru dari awal alur ini.

Bentuk response yang dikirim kembali

- **HTML halaman.** PageController::home() mengembalikan view('home', data); hasil akhirnya biasanya status 200 dengan HTML.
- **Redirect.** BookingController::store() mengembalikan to_route('payments.show', \$payment) dan flash message. Browser menerima redirect, lalu mengirim GET baru ke halaman payment.
- **JSON.** BookingController::availability() dan PaymentController::status() mengembalikan response()->json(...). JavaScript membaca body JSON tanpa memuat ulang seluruh halaman.

- **PDF.** `PaymentController::invoice()` merender view invoice melalui `Dompdf`, lalu mengirim body PDF dengan Content-Type `application/pdf` dan nama file unduhan.

2.3 Apa sebenarnya Blade?

Blade tetap menghasilkan PHP. Sintaks seperti `{{ $product->name }}` mencetak nilai dengan escaping HTML, sedangkan `@foreach` adalah bentuk ringkas perulangan. `@extends` dan `@section` memungkinkan satu layout dipakai banyak halaman. Setelah dirender, browser hanya menerima HTML biasa; browser tidak mengetahui Blade.

Blade HOMCUTS memakai `resources/views/layouts/app.blade.php` untuk halaman pelanggan dan `resources/views/admin/layouts/app.blade.php` untuk admin. Halaman khusus hanya mengisi bagian `title/content`.

Bentuk sederhana sebuah Blade view

```
@extends('layouts.app')

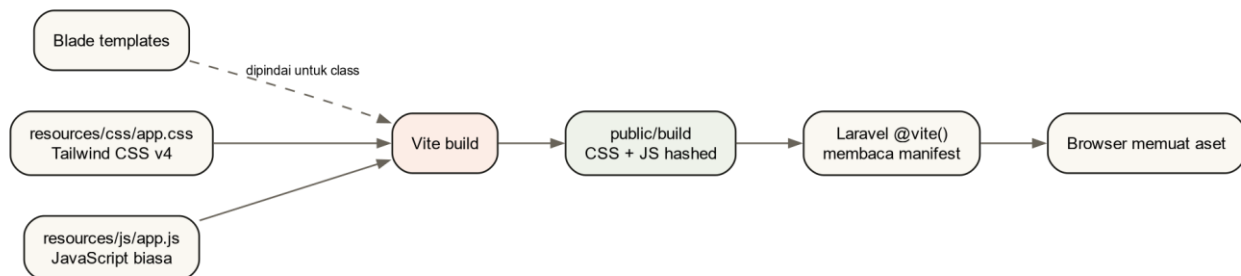
@section('title', 'Galeri')

@section('content')
    @foreach ($entries as $entry)
        <h2>{{ $entry->style }}</h2>
    @endforeach
@endsection
```

2.4 Apa sebenarnya Vite?

Vite bukan framework backend dan tidak menggantikan PHP. Ia adalah alat pengembangan dan build aset frontend. `vite.config.js` mendaftarkan `resources/css/app.css`, `resources/js/app.js`, dan `resources/js/admin-image-editor.js`. Saat build, Vite menghasilkan file bernama hash serta manifest di `public/build`. Direktif `@vite(...)` pada layout membaca manifest agar browser memuat file hasil build yang benar.

Karena Dockerfile menjalankan `npm ci` dan `npm run build` pada tahap assets, server runtime tidak perlu menjalankan Node.js. Node diperlukan ketika membangun image atau mengembangkan aset; halaman yang sudah dibangun tetap dijalankan PHP/Laravel.

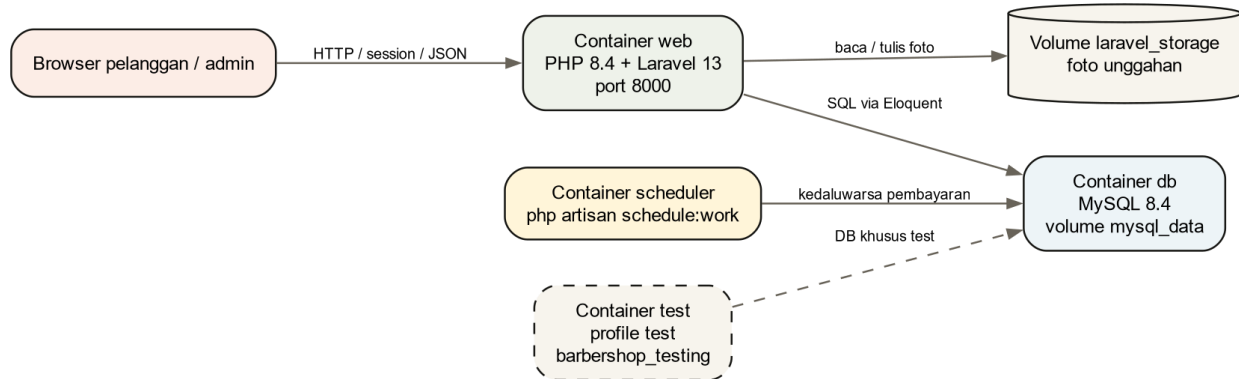


Gambar 2.2 Proses build Blade, Tailwind, JavaScript, dan Vite

BAB 3

Arsitektur Docker dan lingkungan berjalan

Docker memisahkan aplikasi, database, penjadwal, dan pengujian agar lingkungan konsisten.



Gambar 3.1 Arsitektur container dan volume HOMCUTS

3.1 Container db

Service db memakai image `mysql:8.4` dan nama container `homcuts-db`. Ia membuat database utama `barbershop`, user aplikasi `barbershop`, dan menyimpan data di volume `mysql_data`. Healthcheck menjalankan `mysqladmin ping ke 127.0.0.1` dengan `--protocol=TCP`. Compose menunggu pemeriksaan koneksi TCP ini berhasil sebelum web atau scheduler dianggap boleh mulai.

Database tidak dipublikasikan ke port host pada konfigurasi Compose ini. Container lain mengaksesnya melalui nama host `db` di jaringan internal Docker. Karena data berada di volume, docker compose down tidak menghapusnya; docker compose down `--volumes` akan menghapus volume dan harus dipakai dengan sangat hati-hati.

3.2 Container web

Service web menjalankan image hasil Dockerfile, meneruskan port host 8000 ke aplikasi, dan membaca `.env`. Saat mulai, perintah startup membuat storage link, menjalankan migration `--force`, membersihkan compiled view, lalu menjalankan `php artisan serve` pada `0.0.0.0:8000`.

Kode aplikasi sudah dimasukkan ke image ketika build. Volume `laravel_storage` dipasang ke storage agar foto unggahan dan berkas runtime yang perlu bertahan tidak hilang saat container dibuat ulang.

3.3 Container scheduler

Service scheduler memakai image yang sama dengan web tetapi perintahnya `php artisan schedule:work`. Ia tidak melayani browser. Ia menjalankan jadwal pada `routes/console.php`. Saat ini tugas utama adalah `payments:expire` setiap menit dan diberi `withoutOverlapping` agar satu eksekusi tidak menumpuk.

Tanpa scheduler, booking/pesanan tunai yang tidak dibayar dapat tetap menahan slot atau stok sampai halaman tertentu memicu pemeriksaan. Karena itu scheduler adalah “petugas latar belakang” yang menjaga data operasional tetap bersih.

3.4 Service test

Service test berada dalam profile test sehingga tidak hidup pada docker compose up biasa. Ia memakai `APP_ENV=testing` dan database `barbershop_testing`, bukan database utama. Mount source dibuat read-only, sedangkan storage/cache testing diisolasi.

Perintah default container test menjalankan tests/Feature/BlackBoxTest.php. Untuk seluruh suite, command container dapat dioverride menjadi php vendor/bin/phpunit --do-not-cache-result. Service test memakai database barbershop_testing; jangan arahkan RefreshDatabase ke database utama.

3.5 Dockerfile bertahap

Tahap	Image dasar	Apa yang dilakukan	Mengapa dipisah
vendor	composer:2	composer install --no-dev dan optimasi autoload.	Dependency PHP dibangun secara reproduktif.
assets	node:22-alpine	npm ci lalu npm run build.	Node hanya dipakai saat build aset.
app	php:8.4-cli-bookworm	Memasang ekstensi PHP, menyalin source/vendor/build, menyiapkan storage, menjalankan server.	Image runtime lebih bersih dan tidak membawa tool build berlebihan.

Perintah harian dari folder da-project

```
docker compose up --build -d
docker compose ps
docker compose logs -f web
docker compose exec web php artisan migrate:status
docker compose down
```

Bedakan image, container, dan volume

Image adalah cetakan; container adalah proses yang dibuat dari cetakan; volume adalah tempat data bertahan. Menghapus container tidak otomatis menghapus volume.

3.6 Urutan startup web dan kesiapan MySQL

Compose memulai db lebih dahulu dan menunggu status service_healthy sebelum menjalankan web. Setelah MySQL siap, web membuat storage link, menjalankan php artisan migrate --force, membersihkan compiled view, lalu menjalankan php artisan serve. Scheduler menunggu web sehat karena membutuhkan aplikasi dan database yang sama.

Healthcheck saat ini memaksa koneksi TCP: `mysqladmin ping -h 127.0.0.1 --protocol=TCP`. Ini penting saat volume database masih kosong. Image MySQL menyalakan server sementara untuk inialisasi; pemeriksaan lama ke localhost dapat memakai Unix socket dan berhasil sebelum MySQL membuka port TCP 3306. Compose lalu menganggap db sehat terlalu awal. Pemeriksaan TCP menunggu server normal benar-benar menerima koneksi.

Perintah Artisan juga mem-bootstrap aplikasi Laravel. Saat route didaftarkan, `routes/web.php` meminta `AdminResources::keys()`; definisi resource membaca pengaturan lewat `ServiceSchedule::settings()` dan `Schema::hasTable('site_settings')`. Jika MySQL belum menerima koneksi, web dapat gagal sebelum migration dimulai. Dengan healthcheck TCP, bootstrap terjadi setelah koneksi database tersedia.

Error Duplicate column name `image_path` berbeda dari `Connection refused`. Error duplikat muncul bila struktur dalam volume MySQL sudah memiliki kolom yang hendak ditambahkan migration yang masih dianggap pending. docker compose up --build membangun ulang image, tetapi mempertahankan volume `mysql_data`. Karena itu build ulang bukan reset database. Jangan menghapus volume kecuali datanya memang boleh hilang.

BAB 4

Tur halaman pelanggan

Setiap halaman dijelaskan dari tujuan, sumber data, interaksi, dan file yang mengendalikannya.

4.1 Beranda — /

Komponen	Keterangan
Controller	PageController::home
View	resources/views/home.blade.php
Tujuan	Menampilkan hero, produk berjalan, daftar layanan, kartu capster, dan cuplikan galeri.
Data utama	Barber aktif, Service aktif, Product aktif/featured, GalleryEntry published pertama.

4.2 Booking — /booking

Komponen	Keterangan
Controller	PageController::booking
View	resources/views/booking.blade.php
Tujuan	Memilih booking cepat atau capster tertentu, layanan, tanggal, waktu, nama, WhatsApp, dan pembayaran tunai.
Data utama	Capster aktif dan layanan aktif; parameter artist dapat memilih capster dari beranda.

4.3 Toko produk — /shop

Komponen	Keterangan
Controller	PageController::shop
View	resources/views/shop.blade.php + partials/cart.blade.php

Komponen	Keterangan
Tujuan	Menyaring produk, memasukkan ke keranjang, checkout tunai, dan menerima kode transaksi.
Data utama	Product aktif; kategori unik untuk filter.

4.4 Galeri — /gallery

Komponen	Keterangan
Controller	PageController::gallery
View	resources/views/gallery.blade.php
Tujuan	Menampilkan karya, nama model rambut, deskripsi, kredit, dan capster opsional.
Data utama	GalleryEntry published dan relasi capster.

4.5 Tentang kami — /about

Komponen	Keterangan
Controller	Closure route
View	resources/views/about.blade.php
Tujuan	Menjelaskan cerita, nilai, alur layanan, dan foto toko.
Data utama	Pengaturan situs dibagikan global; konten utama bersifat statis.

4.6 Kontak — /contact

Komponen	Keterangan
Controller	Closure + ContactController::store
View	resources/views/contact.blade.php
Tujuan	Menampilkan alamat/sosial/jam dan menerima formulir pesan.
Data utama	site_settings; POST menyimpan contact_messages.

4.7 Status pembayaran — /payments/{reference}

Komponen	Keterangan
Controller	PaymentController::show/status
View	resources/views/payments/show.blade.php
Tujuan	Menampilkan kode transaksi, antrian, jadwal, total, instruksi tunai, status, dan perubahan jadwal.
Data utama	Payment berdasarkan UUID reference, Order, Booking, item, capster, service.

4.8 Layout pelanggan

resources/views/layouts/app.blade.php adalah kerangka bersama. Ia berisi metadata, Open Graph, @vite, navigasi desktop/mobile, tombol keranjang, footer, modal sukses, dan JSON katalog produk untuk JavaScript. Setiap halaman meletakkan isi pada @yield('content').

AppServiceProvider membagikan site_settings dan katalog produk aktif ke layout publik. Karena itu footer dan keranjang dapat dipakai di semua halaman tanpa setiap controller mengulang query yang sama.

4.9 Interaksi tanpa reload penuh

- Menu mobile dibuka/ditutup dengan class hidden dan atribut aria-expanded.
- Slot booking diperiksa 350 ms setelah input berhenti berubah; request lama dibatalkan dengan AbortController.
- Keranjang disimpan di localStorage browser. Database baru berubah ketika checkout dikirim.
- Halaman status memanggil endpoint JSON berkala selama pembayaran/booking masih perlu dipantau.
- Modal dan drawer dapat ditutup dengan tombol, klik backdrop, atau Escape.

BAB 5

Tur panel admin dan kasir

Panel admin dilindungi autentikasi sesi dan middleware hak akses admin.

5.1 Login — /admin/login

AuthController::create menampilkan form jika belum login dan mengarahkan admin aktif ke dashboard. AuthController::store memvalidasi email/password, menjalankan Auth::attempt, meregenerasi session untuk mencegah session fixation, lalu memastikan akun is_admin. Percobaan POST dibatasi lima per menit.

Logout menghapus autentikasi, menginvalidasi session, dan membuat token CSRF baru. Pengguna yang belum login diarahkan ke route login; pengguna login non-admin menerima 403 dari EnsureUserIsAdmin.

5.2 Ringkasan — /admin

DashboardController menghitung pekerjaan tertunda, booking aktif, konfirmasi tunai, pesanan produk, pendapatan hari/bulan, dan transaksi hari ini. Bagian booking aktif dan transaksi terbaru memakai data-live-region sehingga layout admin dapat mengganti HTML region secara berkala tanpa reload seluruh halaman.

Transaksi terbaru menggabungkan booking, walk-in/POS, dan pesanan aplikasi. Query mengecualikan order cancelled dan booking cancelled. Tabel dapat diurutkan dari header secara AJAX.

5.3 Kasir POS — /admin/pos

POS adalah formulir pembuatan transaksi langsung oleh kasir. Ia dapat berisi layanan, produk, atau campuran. Jika memilih layanan, kasir memilih capster dan waktu hari ini. Sistem memakai BookingAvailabilityService yang sama dengan booking pelanggan sehingga jadwal konsisten.

POS hanya berfokus pada transaksi; angka pendapatan berada di dashboard. Setelah disimpan, transaksi tetap belum dibayar sampai tombol konfirmasi uang diterima ditekan.

5.4 Booking dan transaksi

Menu	Fungsi	Catatan
Booking	Melihat, membuat, mengubah jadwal/status, serta konfirmasi pembayaran terkait.	Tidak mendukung hard delete; gunakan cancelled.
Transaksi	Riwayat keuangan terpadu dan konfirmasi pembayaran produk/booking/POS.	Dapat mengubah status proses/catatan; tidak membuat lewat CRUD generik.
Kasir POS	Membuat transaksi walk-in.	Entry point operasional, bukan laporan.

5.5 Data utama

Menu	Yang dikelola	Dampak ke pelanggan
Capster	Foto, nama, slug, spesialisasi, bio, jam kerja, aktif.	Beranda dan pilihan booking.
Layanan	Nama unik, slug, harga, deskripsi, urutan, aktif. Durasi operasional diatur global.	Daftar layanan, booking, POS.
Produk	Foto, nama, kategori, harga, stok, ukuran, badge, aktif.	Beranda, toko, keranjang.
Galeri	Foto upload, nama model, kredit, capster opsional, fokus, deskripsi.	Galeri; tidak membuat pilihan layanan.

Menu	Yang dikelola	Dampak ke pelanggan
Pesan	Status dan catatan tindak lanjut.	Data berasal dari form kontak.
Pengaturan	Key/value informasi umum, kontak, sosial, jam buka/tutup, durasi layanan global, dan batas pembayaran.	Footer, kontak, jadwal booking, dan tenggat transaksi.

5.6 Resource CRUD generik

ResourceController dan AdminResources membentuk mesin CRUD generik. AdminResources adalah konfigurasi: model, tabel, kolom, field, aturan validasi, relasi, dan izin create/delete. ResourceController membaca konfigurasi yang sama untuk list, form, simpan, update, upload, filter, sort, dan pagination.

Keuntungan pendekatan ini adalah konsistensi dan pengurangan duplikasi. Risiko utamanya: perubahan satu aturan generik bisa memengaruhi banyak halaman, sehingga pengujian admin harus dijalankan setelah modifikasi.

5.7 Akun saya

AccountController memungkinkan admin memperbarui nama/email dan kata sandi. Email harus unik. Pergantian kata sandi mensyaratkan password saat ini, konfirmasi password, minimal delapan karakter, serta kombinasi huruf dan angka.

BAB 6

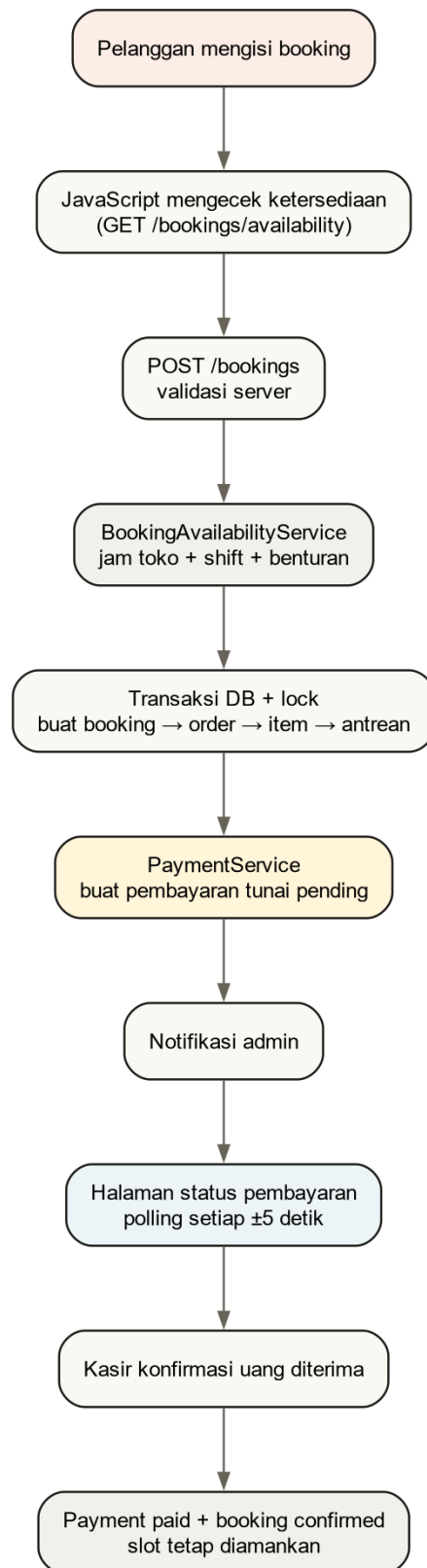
Workflow booking secara mendalam

Booking adalah gabungan validasi jadwal, transaksi database, nomor antrean, pembayaran tunai, dan pembaruan status.

6.1 Pemeriksaan awal di browser

app.js membaca service, tanggal, waktu, mode booking, dan capster. Nilai jam buka, jam tutup, serta durasi global berasal dari pengaturan situs. Waktu mulai terakhir dihitung dari jam tutup dikurangi durasi layanan. Jam kerja capster dapat membatasi pilihan lebih jauh. Nilai min/max pada form hanya membantu pengguna; server tetap melakukan validasi penuh.

Request GET /bookings/availability menghasilkan JSON. Jika tidak tersedia, pesan merah muncul dan tombol submit dinonaktifkan. Jika request gagal karena jaringan, submit tetap diizinkan karena controller akan memeriksa ulang dengan aturan yang sama.



Gambar 6.1 Urutan booking pelanggan sampai pembayaran dikonfirmasi

6.2 Validasi BookingController

1. Memastikan booking_type adalah service atau artist.

2. Memastikan service aktif dan, jika dipilih, capster aktif.
3. Memastikan format tanggal/waktu, nama, telepon, dan metode cash benar.
4. Menolak current minute atau waktu lampau.
5. Memproses pembayaran kedaluwarsa lebih dahulu agar slot lama dapat dilepas.

6.3 Algoritma ketersediaan

BookingAvailabilityService membaca service_duration_minutes, store_open_time, dan store_close_time melalui ServiceSchedule. Nilai awalnya 45 menit, 07.00, dan 22.00, tetapi admin dapat mengubahnya di Pengaturan. Waktu mulai terakhir adalah jam tutup dikurangi durasi global. Jadwal juga harus berada di dalam work_start_time dan work_end_time capster.

Benturan memakai aturan interval: existing_start < new_end DAN existing_end > new_start. Dua jadwal yang hanya bersentuhan pada batas diperbolehkan; misalnya 10.00–10.30 dan 10.30–11.00 tidak bertabrakan. Pemeriksaan mencakup booking maupun order POS berkategori layanan/campuran yang belum cancelled.

Rumus benturan interval

```
existing.start < requested.end
AND existing.end > requested.start
```

6.4 Booking cepat dan capster tertentu

Mode service berarti capster mana saja. Service mengurutkan capster aktif dan memilih yang pertama yang jam kerjanya cocok serta tidak bentrok. Mode artist membatasi kandidat pada capster yang dipilih. Keduanya tetap memakai aturan yang sama.

6.5 Transaksi atomik dan snapshot

Pembuatan booking memakai DB::transaction dan lockForUpdate agar dua request bersamaan tidak mengambil slot yang sama. Setelah Booking dibuat, BookingTransactionService membuat Order channel booking, satu OrderItem bertipe service, snapshot nama/harga/durasi/capster, dan nomor antrean.

Snapshot berarti transaksi lama tidak berubah hanya karena harga katalog diubah. Setelah payment attempt ada, perubahan layanan/harga dibatasi. Status-only update mempertahankan total, durasi, dan nilai payment.

6.6 Pembayaran, tenggat, dan perubahan jadwal

PaymentService membuat Payment cash/pending dengan UUID reference dan batas waktu yang dibaca PaymentPolicy. Nilai payment_expiry_minutes di Pengaturan berlaku untuk booking dan pesanan produk; bila pengaturan belum tersedia, konfigurasi memakai nilai fallback 30 menit. Nilai dibatasi antara 5 dan 1.440 menit. Booking juga menyimpan hold_expires_at. Jika pembuatan payment gagal, order dibatalkan agar booking tidak menjadi orphan yang memblokir slot.

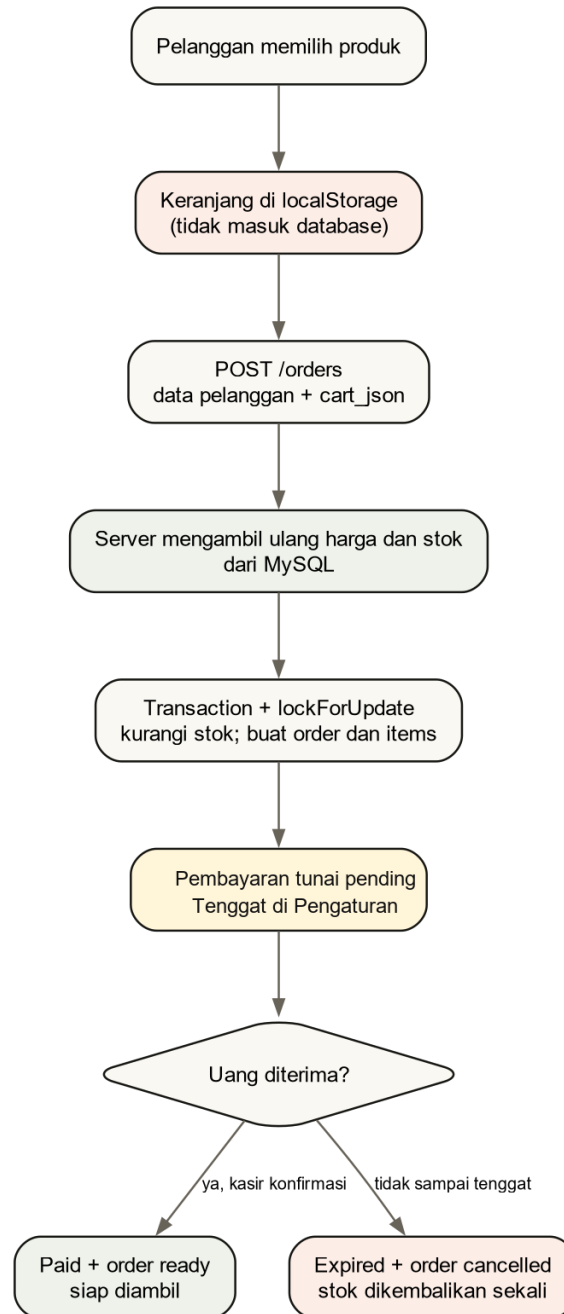
Ketika kasir mengonfirmasi uang, payment menjadi paid, order payment_status menjadi paid, booking menjadi confirmed, dan hold_expires_at dikosongkan. Jika tenggat habis lebih dahulu, scheduler mengubah payment expired, order/booking cancelled, lalu slot dapat dipakai lagi.

Perubahan jadwal admin mencatat schedule_changed_at dan otomatis tampil pada halaman status pelanggan tanpa refresh manual.

BAB 7

Workflow toko produk

Keranjang berada di browser; stok dan harga tetap ditentukan server saat checkout.



Gambar 7.1 Urutan pembelian produk dan pengelolaan stok

7.1 Keranjang localStorage

Saat tombol Tambah ditekan, app.js membaca data produk yang ditanam pada tombol, memasukkan item ke array cart, lalu menyimpan JSON ke localStorage dengan key homcuts-cart. Ini membuat keranjang bertahan ketika halaman dipindah atau browser direfresh.

Karena localStorage bisa dimanipulasi pengguna, nilainya tidak dipercaya sebagai harga/stok. Browser hanya mengirim id dan quantity melalui cart_json.

7.2 Checkout OrderController

1. Validasi identitas pelanggan, metode cash, dan bentuk JSON cart.
2. Gabungkan item duplikat dan batasi quantity per produk.
3. Ambil Product aktif dari MySQL dengan lockForUpdate.
4. Tolak jika produk hilang/nonaktif atau quantity melebihi stok.
5. Hitung ulang subtotal dari harga database; jangan memakai total browser.
6. Kurangi stok dan buat Order channel online, transaction_type product, OrderItem snapshot.
7. **Buat Payment cash pending dengan batas waktu dari PaymentPolicy dan kirim notifikasi admin. Batas waktu yang sama diatur melalui Pengaturan situs dan dipakai untuk booking maupun pesanan produk.**

7.3 Konfirmasi dan pengambilan

Pelanggan membawa kode transaksi dan membayar tunai. Admin mengonfirmasi dari halaman Transaksi. PaymentService mengubah payment.status menjadi paid dan order.payment_status menjadi paid. Status proses order biasanya tetap pending, sehingga antarmuka menampilkan bahwa pesanan sudah dibayar tetapi masih menunggu diambil. Setelah barang diserahkan, admin menandai status proses completed.

Jika transaksi kadaluarsa, cancelLockedOrder mengembalikan stok tepat satu kali menggunakan stock_released_at sebagai tanda idempotensi. Menjalankan scheduler berulang tidak menambah stok berkali-kali.

Mengapa stok dikurangi saat checkout?

Stok “diamankan” agar dua pelanggan tidak membeli unit yang sama. Tenggat dan scheduler mencegah stok tertahan selamanya ketika pelanggan tidak datang membayar.

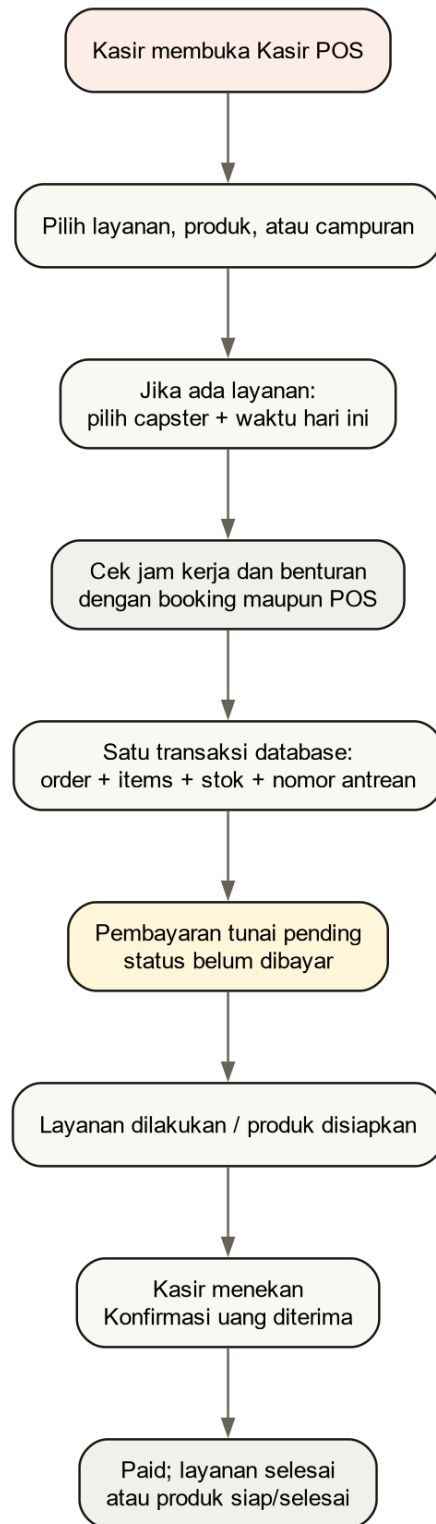
BAB 8

POS, antrean, transaksi, dan pembayaran

Booking dan walk-in berbeda pintu masuk, tetapi akhirnya memakai struktur transaksi, antrean, jadwal, dan pembayaran yang sama.

8.1 Tipe transaksi

channel	transaction_type	Makna
booking	service	Booking yang dibuat pelanggan/admin dan ditautkan ke bookings.
cashier	service	Walk-in layanan dari POS.
cashier	product	Penjualan produk langsung di kasir tanpa antrean layanan.
cashier	mixed	Layanan + produk dalam satu transaksi POS.
online	product	Pesanan produk dari situs untuk diambil.



Gambar 8.1 Workflow transaksi walk-in melalui Kasir POS

8.2 Jadwal terpadu booking dan POS

Untuk POS dengan layanan, PosController memakai BookingAvailabilityService. Slot dihitung untuk tanggal hari ini dan menolak waktu lampau. Konflik dicek terhadap booking serta order POS service/mixed. Dengan demikian, jadwal 10.00–10.30 dari POS membuat booking capster yang sama baru tersedia mulai 10.30, dan sebaliknya.

Kolom `orders.service_starts_at` dan `service_ends_at` menyimpan jadwal layanan POS. Booking juga mempunyai `starts_at/ends_at`, sedangkan order booking menyalin jadwal tersebut untuk laporan terpadu.

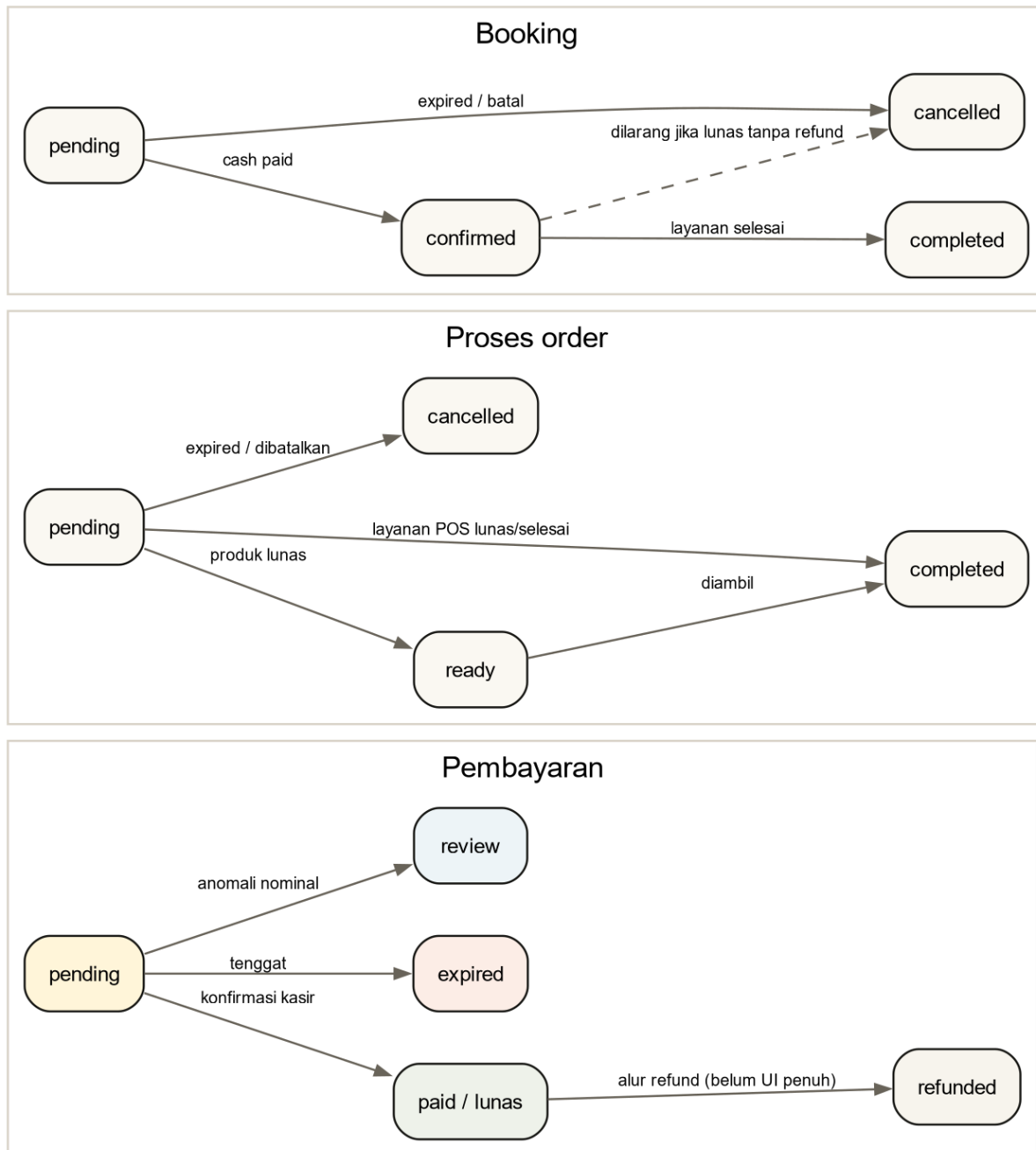
8.3 Nomor antrean global

`QueueNumberService` mengunci satu baris `queue_sequences` bernama `service`, menaikkan `last_number`, lalu mengisi `orders.queue_number`. Karena booking dan POS memanggil `service` yang sama, nomor tidak duplikat. `queue_code` accessor menampilkan A diikuti tiga digit, misalnya A002.

Nomor antrean tidak diberikan pada transaksi `product-only` karena tidak membutuhkan giliran cukur. Migration lama `daily_queue_sequences` masih ada sebagai jejak evolusi skema, tetapi logika aktif memakai `queue_sequences` global.

8.4 Status proses versus status pembayaran

`Order.status` menjawab “pekerjaan/pesanan sudah sampai tahap mana?”, sedangkan `order.payment_status` menjawab “uang sudah diterima atau belum?”. Pemisahan ini diperlukan: produk dapat sudah lunas tetapi masih `ready` (belum diambil), atau booking dapat `confirmed` tetapi layanan belum `completed`.



Gambar 8.2 Transisi status pembayaran, order, dan booking

8.5 PaymentService dan idempotensi

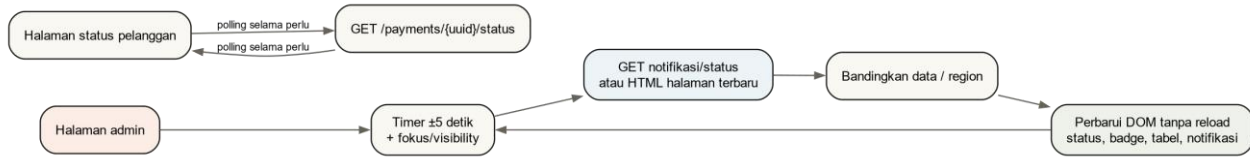
PaymentService adalah pusat lifecycle pembayaran. Method `createForOrder`, `markPaid`, `confirmCash`, `expire`, `expireDuePayments`, `cancelOrder`, dan `fail` menggunakan transaksi dan row lock. Operasi penting dibuat idempotent: konfirmasi paid kedua tidak membuat pembayaran ganda; expiry kedua tidak mengembalikan stok dua kali.

Sebelum paid, nominal payment dibandingkan dengan `order.total`. Ketidaksesuaian diarahkan ke review dan tidak melunasi order. Satu order tidak boleh mempunyai dua pembayaran paid. Pembatalan order yang sudah lunas ditolak sampai tersedia alur refund yang benar.

BAB 9

Notifikasi dan pembaruan otomatis

Aplikasi memakai polling ringan dan penggantian bagian DOM, bukan WebSocket.



Gambar 9.1 Mekanisme live update pada pelanggan dan admin

9.1 Notifikasi database

AdminNotifier mencari seluruh User is_admin lalu mengirim AdminActivityNotification melalui channel database. Laravel menulis record ke tabel notifications berisi type, notifiable, data JSON, read_at, dan waktu. Contoh pemicu: booking baru, pesanan produk baru, dan pembayaran diterima.

NotificationController::index mengembalikan unread_count, sepuluh notifikasi terbaru, dan status booking/order yang diminta halaman. Endpoint read/readAll hanya dapat memodifikasi notifikasi milik admin yang sedang login.

9.2 Polling admin

resources/views/admin/layouts/app.blade.php berisi JavaScript polling sekitar lima detik. Ia tidak hanya memperbarui ikon notifikasi, tetapi juga badge pembayaran/status dan region HTML bertanda data-live-region. Polling dihentikan sementara ketika tab tidak terlihat dan dipicu lagi ketika halaman kembali fokus.

Konfirmasi pembayaran dilakukan via fetch POST. Setelah server berhasil, elemen terkait diperbarui tanpa reload. Sorting header juga mengambil HTML terbaru, mencari region dengan nama yang sama, lalu menggantinya dan memperbarui URL menggunakan history.replaceState.

9.3 Polling pelanggan

Halaman payment status hanya polling ketika payment pending atau booking masih dapat berubah. Endpoint /payments/{reference}/status mengirim status payment, order, serta informasi booking terbaru. Jika response 429, jeda dinaikkan menjadi 30 detik; jika jaringan gagal, menjadi 10 detik.

Mengapa bukan “realtime” murni?

Polling terasa otomatis tanpa menambah Redis, WebSocket server, atau layanan broadcast. Trade-off-nya: perubahan terlihat setelah jeda beberapa detik dan menghasilkan request berkala.

MySQL menyimpan data operasional, sedangkan Eloquent memetakan tabel ke object PHP.



Tabel	Tujuan	Kolom/aturan penting
users	Akun admin/kasir dan flag is_admin.	email unique; password hash.
barbers	Data capster. Nama tabel tetap historis “barbers”, UI memakai Capster.	slug unique; work hours; image.
services	Katalog layanan.	name/slug unique; price; durasi 45 menit menjadi data legacy. Durasi aktif dibaca dari site_settings global.
products	Katalog dan stok produk.	slug unique; price; stock; active.
gallery_entries	Foto karya dan deskripsi.	barber_id opsional; published.
bookings	Reservasi layanan pelanggan.	barber/service FK; interval; hold; status.
orders	Sumber laporan transaksi terpadu.	booking/cashier FK; queue; schedule; totals; statuses.
order_items	Rincian produk/layanan pada order.	Snapshot nama, harga, qty; optional product/service/capster FK.
payments	Percobaan dan status pembayaran.	UUID reference; amount; method; expiry; confirmer.
contact_messages	Pesan form kontak.	Status follow-up dan catatan.
site_settings	Konfigurasi situs dan operasional.	key unique, value, group; termasuk jam buka/tutup, durasi layanan global, dan batas pembayaran.
notifications	Notifikasi admin Laravel.	UUID, JSON data, read_at.
queue_sequences	Counter antrean global.	name=service; last_number.

Model	Relasi	Makna
Booking	belongsTo Barber; belongsTo Service; hasOne transaction	Satu booking punya capster/layanan dan satu order aktif.
Order	hasMany items/payments; belongsTo booking/cashier	Satu transaksi memiliki rincian dan riwayat payment attempt.
OrderItem	belongsTo product/service/barber	Satu baris dapat mewakili produk atau layanan.
Barber	hasMany bookings/galleryEntries	Capster dapat muncul di jadwal dan galeri.

Model	Relasi	Makna
Payment	belongsTo order/confirmer	Pembayaran milik order dan dapat dikonfirmasi admin.

10.3 Migration bukan query harian

Migration mengubah struktur atau data awal database. Tabel migrations mencatat migration yang sudah berhasil dijalankan. Saat container web mulai, artisan migrate --force memeriksa daftar tersebut dan menjalankan migration yang masih pending. Status aktual dapat dilihat dengan php artisan migrate:status; jangan menganggap semua environment memiliki status yang sama.

Jangan mengedit migration lama yang sudah dipakai produksi untuk perubahan baru. Buat migration baru agar database lama dapat dinaikkan secara aman dan riwayat perubahan tetap jelas.

10.4 Transaksi, lock, dan foreign key

DB::transaction memastikan serangkaian perubahan sukses bersama atau rollback bersama. lockForUpdate mencegah request lain mengubah baris yang sedang dihitung dalam transaksi. Keduanya penting untuk stok, slot, payment, dan nomor antrean.

Foreign key menjaga referensi. Banyak relasi katalog memakai nullOnDelete agar transaksi lama tetap ada meski data master dihapus; order_items pada order memakai cascadeOnDelete. Namun panel admin sengaja menghindari hard delete untuk booking/order agar audit keuangan tidak hilang.

BAB 11

Membaca struktur kode

Bagian ini menjelaskan folder dan bagaimana menentukan file yang harus dibuka ketika ingin memahami atau mengubah fitur.

11.1 Struktur folder utama

Peta folder proyek

```

da-project/
├── compose.yaml      orkestrasi container
├── Dockerfile        cara membangun image aplikasi
├── .dockerignore     berkas yang tidak masuk build context
├── barbershop-laravel/ source Laravel
│   ├── app/          kode bisnis PHP
│   ├── bootstrap/    bootstrap framework
│   ├── config/        konfigurasi
│   ├── database/      migration, seeder, factory
│   ├── public/        document root dan aset publik
│   ├── resources/     Blade, CSS, JavaScript
│   ├── routes/        daftar endpoint/command terjadwal
│   ├── storage/       upload, log, cache runtime
│   ├── tests/         PHPUnit
│   ├── vendor/        dependency Composer (generated)
│   └── node_modules/  dependency npm (generated)
├── output/           dokumen dan bukti hasil kerja
└── tmp/              berkas sementara pembuatan artefak
  
```

11.2 Folder app

Folder	Isi	Kapan dibuka
app/Http/Controllers	Penerima request publik/admin.	Saat mencari validasi, redirect, atau response fitur.
app/Http/Middleware	Filter akses.	Saat memeriksa login/admin.
app/Models	Representasi tabel dan relasi.	Saat mencari fillable, cast, relasi, accessor.
app/Services	Aturan bisnis kompleks.	Slot, sinkronisasi booking-order, pembayaran, antrean, notifikasi.
app/Support	Konfigurasi pendukung non-request.	Definisi CRUD generik admin.
app/Notifications	Format notifikasi.	Saat mengubah data notifikasi database.
app/Providers	Bootstrap/registrasi global.	Saat data dibagikan ke semua view.

11.3 Cara mencari kode sebuah fitur

1. Cari URL pada routes/web.php.
2. Lihat controller dan method tujuan.
3. Ikuti service yang dipanggil untuk aturan bisnis.
4. Lihat model/relasi yang di-query atau disimpan.
5. Cari view pada return view(...) atau response JSON.
6. Jika tampilan interaktif, cari id/data-* pada resources/js/app.js atau layout admin.
7. Cari test yang menyebut route/fitur untuk memahami perilaku yang diharapkan.

11.4 Contoh pelacakan “konfirmasi uang diterima”

Jejak fitur dari tombol sampai database

```
routes/web.php
→ Admin\PaymentController::confirmCash
→ PaymentService::confirmCash
→ PaymentService::markPaid
→ update payments + orders + bookings
→ AdminNotifier
→ JSON response
→ JavaScript admin memperbarui badge/tabel
```

11.5 File aktif, generated, dan legacy

Kategori	Contoh	Perlakuan
Source aktif	app/, resources/, routes/, config/, database/migrations	Boleh dibaca/diubah dengan test.
Generated/dependency	vendor/, node_modules/, public/build/, bootstrap/cache/	Jangan diedit manual; buat ulang dengan Composer/npm/Artisan.
Runtime	storage/framework, storage/logs, upload storage	Bukan source; backup upload bila penting.
Legacy tidak aktif	MidtransGateway, MidtransWebhookController, welcome.blade.php, daily_queue_sequences	Jangan diasumsikan aktif; route pembayaran QRIS sudah tidak terdaftar.
Tidak dipakai Docker utama	database/database.sqlite	Docker menggunakan MySQL.
		Nama internal “Barber” UI sudah memakai istilah Capster, tetapi class Barber, tabel barbers, barber_id, dan beberapa variable JavaScript tetap dipertahankan agar migrasi besar tidak merusak data. Ini bukan bug tampilan; ini kompatibilitas internal.

BAB 12

Frontend: Blade, Tailwind, dan JavaScript

Frontend proyek dibuat tanpa framework SPA; HTML server-rendered diperkaya JavaScript biasa.

12.1 Dua layout utama

Layout	Dipakai oleh	Isi penting
layouts/app.blade.php	Semua halaman pelanggan	Navbar, mobile menu, cart drawer, checkout, footer, modal, @vite, katalog JSON.
admin/layouts/app.blade.php	Semua halaman admin	Sidebar, header, notifikasi, CSRF, polling live, sorting AJAX, konfirmasi cash.

12.2 Tailwind CSS v4

app.css mengimpor tailwindcss, mendefinisikan @source agar Vite memindai Blade/JS, serta membuat tema warna ink, muted, cream, paper, orange, dan sage. @layer components mendefinisikan class berulang seperti nav-link, btn-primary, form-control, booking-tab, filter-button, product-marquee, dan tabel admin.

Responsive admin table memakai media query: pada layar di bawah 1280 px, tabel berubah menjadi kumpulan kartu dua kolom, lalu satu kolom pada layar ponsel. prefers-reduced-motion menghentikan animasi marquee bagi pengguna yang meminta pengurangan gerak.

12.3 Peta fungsi resources/js/app.js

Blok	Tanggung jawab	Data penting
Menu mobile	Toggle hidden dan ikon.	aria-expanded.
Mode booking	Booking cepat vs pilih capster.	booking_type, artist_id.
Availability	Debounce 350 ms, AbortController, pesan slot.	service/date/time/artist.
Filter toko	Sembunyikan kartu tanpa request server.	data-category.
Cart	localStorage, quantity, total, drawer, checkout.	homcuts-cart, cart_json.
Payment status	Countdown dan polling endpoint status.	data-status-url, expiresAt.
Accessibility	Escape, focus, aria.	Modal/menu/drawer.

12.4 Keamanan output dan input

Blade {{ ... }} melakukan HTML escaping. JavaScript cart juga mempunyai escapeHtml sebelum menanam teks ke innerHTML. Semua form POST menyertakan CSRF token. Meski browser mempunyai validasi min/max, server tetap memvalidasi semua input.

BAB 13

Keamanan dan konsistensi data

Lapisan keamanan proyek bukan hanya login; aturan server juga melindungi harga, stok, slot, dan perubahan status.

13.1 Autentikasi dan otorisasi

- Password disimpan menggunakan hash Laravel, bukan teks biasa.
- Session diregenerasi setelah login dan diinvalidasi saat logout.
- Middleware auth memastikan sesi ada; middleware admin memastikan is_admin.
- CSRF melindungi request perubahan data dari situs lain.
- Login, booking, checkout, dan status polling diberi throttle sesuai risiko.

13.2 Integritas transaksi

- Harga checkout dihitung ulang dari database.
- Produk dikunci saat stok diperiksa/dikurangi.
- Slot capster dikunci saat booking/POS disimpan.
- Nomor antrean diambil dengan lock counter.
- Payment paid harus cocok nominalnya dengan order.total.
- Satu order tidak boleh dibayar dua kali oleh payment berbeda.
- Pembatalan order lunas ditolak sampai tersedia proses refund.

13.3 Upload foto

ResourceController menerima JPG/JPEG/PNG/WebP maksimal 5 MB untuk produk, capster, dan galeri. Berkas disimpan pada disk public di uploads/{table}. storage:link membuat public/storage menunjuk ke storage/app/public. Saat foto diganti atau record terkelola dihapus, berkas upload lama dibersihkan.

13.4 Risiko operasional yang perlu dipahami

Risiko	Perlindungan saat ini	Praktik yang disarankan
Database utama terhapus saat test	Service test memakai barbershop_testing.	Jangan override ke DB_DATABASE=barbershop; periksa APP_ENV.
Stok/slot tertahan	Expiry + scheduler + compensating cancel.	Pastikan scheduler hidup.
Admin default tidak aman	Credential dari .env.	Ganti password; jangan commit .env.
APP_DEBUG di publik	Saat ini contoh local true.	Produksi: APP_ENV=production, APP_DEBUG=false.
Backup	Volume mempertahankan data lokal.	Tetap buat dump MySQL dan backup storage upload.

BAB 14

Pengujian dan cara membaca test

Test adalah contoh perilaku yang dapat dijalankan; ia membantu memahami sistem sekaligus mencegah regresi.

14.1 Mengapa database testing terpisah

Banyak test memakai RefreshDatabase. Trait ini menjalankan/mengulang migration sehingga data test dimulai bersih dan setiap test terisolasi. Jika diarahkan ke database utama, data utama berisiko hilang. Compose service test sudah mengatur DB_DATABASE=barbershop_testing.

APP_ENV=testing memberi tahu Laravel bahwa proses sedang menguji. Cache/session menggunakan driver array agar cepat dan tidak bercampur dengan situs aktif. Test berjalan di container sekali pakai dan berhenti setelah selesai.

14.2 Kelompok pengujian

File	Fokus	Contoh perilaku
BlackBoxTest.php	16 skenario utama dari perspektif pengguna.	Jam operasional, konflik, checkout, payment, login, POS, CRUD.
ErpWorkflowTest.php	Aturan ERP dan edge case.	Antrean unik, expiry, amount mismatch, paid cannot cancel, schedule snapshot.
AdminTest.php	Akses dan CRUD admin.	Redirect/403, sorting, upload, duplicate service, navigasi.
BarbershopTest.php	Halaman publik.	Brand, halaman terpisah, modal, booking, total server, kontak.
PosTest.php	Operasional POS.	Unpaid sampai konfirmasi, shared schedule, mixed/product queue behavior.
ExampleTest.php	Smoke test bawaan.	Response dan unit true.

14.3 Cara membaca sebuah test

1. Arrange: buat data awal/factory dan set waktu bila perlu.
2. Act: kirim get/post/put/delete atau panggil service.
3. Assert: periksa response, redirect, error validasi, database, dan relasi.
4. RefreshDatabase membersihkan konteks antar test.

Pola konseptual test black-box

```
public function test_booking_di_luar_jam_ditolak(): void
{
    // Arrange: buat capster dan layanan aktif
    // Act: POST booking sebelum 07:00
    // Assert: session memiliki error appointment_time
    //         database tidak memiliki booking baru
}
```

14.4 Menjalankan pengujian

Suite	Cakupan	Cara menjalankan
Black-box	Request/response publik dan admin, database, validasi, booking, stok, pembayaran.	docker compose --profile test run --rm test

Suite	Cakupan	Cara menjalankan
Seluruh suite	Feature test dan unit test.	Override command service test ke php vendor/bin/phpunit --do-not-cache-result; gunakan barbershop_testing.

Contoh perintah pengujian dari root proyek

```
docker compose --profile test run --rm test

# Seluruh suite:
docker compose --profile test run --rm test \
  php vendor/bin/phpunit --do-not-cache-result
```

BAB 15

Panduan perubahan dan diagnosis

Gunakan alur yang terukur agar perubahan tidak merusak booking, stok, payment, dan laporan.

15.1 Siklus perubahan yang aman

1. Definisikan perilaku yang ingin diubah dari perspektif pengguna.
2. Temukan route, controller/service, model, dan view/JavaScript terkait.
3. Periksa relasi status dan efek samping: stok, slot, payment, antrean, notifikasi.
4. Tambahkan/ubah migration jika struktur database berubah; jangan mengedit migration lama yang sudah ran.
5. Ubah kode sekecil mungkin dan jaga transaksi/lock.
6. Jalankan test relevan lalu seluruh suite.
7. Build ulang image/aset dan verifikasi halaman pada ukuran desktop/mobile.
8. Backup data sebelum migrasi/perubahan produksi.

15.2 Peta diagnosis cepat

Gejala	Periksa pertama	Lalu periksa
Halaman 500	<code>docker compose logs -f web</code>	storage/logs, stack trace, .env, migration.
DB connection refused	<code>docker compose ps</code> ; health db	DB_HOST=db, user/password, jaringan Compose.
CSS/JS lama	Build image/Vite	public/build manifest, browser cache.
Foto tidak muncul	storage link dan path	volume laravel_storage, permission, record image_path.
Slot seharusnya kosong tetapi ditolak	booking/order non-cancelled pada interval	payment orphan/expired dan scheduler.
Status tidak berubah otomatis	Network request polling/status	429, tab hidden, data-live-region.
Stok salah	order cancelled/stock_released_at	payment expiry dan item quantity.
Antrean duplikat	queue_sequences dan unique index	semua jalur memakai QueueNumberService.

15.3 Perintah pemeliharaan yang aman

Diagnosis tanpa mengubah data bisnis

```
docker compose ps
docker compose logs -f web
docker compose exec web php artisan about
docker compose exec web php artisan route:list
docker compose exec web php artisan migrate:status
docker compose exec web php artisan optimize:clear
```

Perintah berbahaya

Jangan menjalankan `migrate:fresh`, `db:wipe`, atau `docker compose down --volumes` pada database utama kecuali benar-benar ingin menghapus data dan sudah mempunyai backup.

15.4 Checklist mampu menjelaskan proyek

- Saya dapat menjelaskan route → controller → service → model → Blade/JSON.
- Saya dapat menjelaskan mengapa Blade bukan browser framework dan mengapa Vite bukan backend.
- Saya dapat membedakan order.status, order.payment_status, booking.status, dan payment.status.
- Saya dapat menjelaskan satu jadwal terpadu untuk booking/POS dan rumus benturannya.
- Saya dapat menjelaskan localStorage cart versus data MySQL saat checkout.
- Saya dapat menjelaskan container web, db, scheduler, test, dan fungsi volume.
- Saya tahu test tidak boleh diarahkan ke database utama.
- Saya tahu file aktif, generated, runtime, dan legacy.

BAB LAMPIRAN A

Katalog route

Daftar endpoint utama dan tanggung jawabnya.

Metode	Path	Tujuan kode	Fungsi
GET	/	PageController@home	Beranda publik
GET	/booking	PageController@booking	Form booking
GET	/shop	PageController@shop	Katalog produk
GET	/gallery	PageController@gallery	Galeri
GET	/about	Closure view	Tentang kami
GET	/contact	Closure view	Kontak
POST	/contact	ContactController@store	Simpan pesan
POST	/bookings	BookingController@store	Buat booking; throttle 10/mnt
GET	/bookings/availability	BookingController@availability	Cek slot; throttle 120/mnt
POST	/orders	OrderController@store	Checkout produk; throttle 10/mnt
GET	/payments/{payment}	PaymentController@show	Status/struk transaksi
GET	/payments/{payment}/status	PaymentController@status	JSON live status; throttle 120/mnt
GET	/admin/login	Admin\AuthController@create	Form login
POST	/admin/login	Admin\AuthController@store	Login; throttle 5/mnt
POST	/admin/logout	Admin\AuthController@destroy	Logout
GET	/admin	Admin\DashboardController	Ringkasan
GET/PUT	/admin/account	Admin\AccountController	Akun admin
GET/POST	/admin/pos	Admin\PosController	POS dan simpan transaksi
GET	/admin/pos/availability	Admin\PosController@availability	Cek slot POS
GET	/admin/notifications	Admin\NotificationController@index	JSON notifikasi/live state
POST	/admin/notifications/{id}/read	Admin\NotificationController@read	Tandai dibaca
POST	/admin/notifications/read-all	Admin\NotificationController@readAll	Tandai semua
POST	/admin/orders/{order}/confirm-cash	Admin\PaymentController@confirmCash	Konfirmasi uang tunai
GET	/admin/bookings/availability	Admin\ResourceController@bookingAvailability	Cek slot admin
GET	/admin/{resource}	Admin\ResourceController@index	Daftar resource
GET	/admin/{resource}/create	Admin\ResourceController@create	Tampilkan form resource
GET	/admin/{resource}/{record}/edit	Admin\ResourceController@edit	Tampilkan form edit
DELETE	/admin/{resource}/{record}	Admin\ResourceController@destroy	Hapus jika diizinkan
POST	/admin/{resource}	Admin\ResourceController@store	Simpan resource baru
GET	/payments/{payment}/invoice.pdf	PaymentController@invoice	Unduh invoice PDF

Metode	Path	Tujuan kode	Fungsi
POST	/admin/orders/{order}/confirm-deposit	Admin\PaymentController@confirmDeposit	Konfirmasi DP booking
POST	/admin/orders/{order}/complete	Admin\PaymentController@complete	Selesaikan transaksi/layanan

Semua route admin operasional

Selain login, route admin berada dalam middleware auth dan admin. Placeholder {payment} memakai UUID reference karena model Payment mengubah route key.

BAB LAMPIRAN B

Inventaris folder dan file

Inventaris source, konfigurasi, data awal, view, aset, dan test. Dependency/generated besar dijelaskan sebagai kelompok.

B.1 Folder root da-project

Path	Fungsi
Dockerfile	Resep image multi-stage Composer, Vite, dan PHP runtime.
compose.yaml	Mendefinisikan service db, web, scheduler, test, network implisit, dan volume.
.dockerignore	Mengecilkan build context dan mencegah file lokal tak perlu masuk image.
barbershop-laravel/	Aplikasi Laravel utama.
output/	Dokumen/bukti yang dihasilkan; bukan runtime aplikasi.
tmp/	Berkas kerja sementara; bukan runtime aplikasi.

B.2 File aplikasi

File	Peran
`.editorconfig`	Aturan format dasar editor.
`.env`	Konfigurasi lokal dan rahasia; jangan commit atau tampilkan isinya.
`.env.example`	Template environment tanpa rahasia aktual.
`.gitattributes`	Aturan atribut file Git.
`.gitignore`	Berkas/folder yang tidak dilacak Git.
`.npmrc`	Konfigurasi npm proyek.
`.README.md`	Panduan singkat menjalankan Docker, akun, alur, pembayaran, dan test.
`app/Http/Controllers/Admin/AccountController.php`	Controller panel admin: Account.
`app/Http/Controllers/Admin/AuthController.php`	Controller panel admin: Auth.
`app/Http/Controllers/Admin/DashboardController.php`	Controller panel admin: Dashboard.
`app/Http/Controllers/Admin/NotificationController.php`	Controller panel admin: Notification.
`app/Http/Controllers/Admin/PaymentController.php`	Controller panel admin: Payment.
`app/Http/Controllers/Admin/PosController.php`	Controller panel admin: Pos.
`app/Http/Controllers/Admin/ResourceController.php`	Controller panel admin: Resource.
`app/Http/Controllers/BookingController.php`	Controller publik: Booking.
`app/Http/Controllers/ContactController.php`	Controller publik: Contact.
`app/Http/Controllers/Controller.php`	Controller publik: .
`app/Http/Controllers/MidtransWebhookController.php`	Controller publik: MidtransWebhook.
`app/Http/Controllers/OrderController.php`	Controller publik: Order.
`app/Http/Controllers/PageController.php`	Controller publik: Page.
`app/Http/Controllers/PaymentController.php`	Controller publik: Payment.

File	Peran
`app/Http/Middleware/EnsureUserIsAdmin.php`	Middleware pemeriksaan hak akses admin.
`app/Models/Barber.php`	Model Eloquent untuk entitas/tabel Barber.
`app/Models/Booking.php`	Model Eloquent untuk entitas/tabel Booking.
`app/Models/ContactMessage.php`	Model Eloquent untuk entitas/tabel ContactMessage.
`app/Models/GalleryEntry.php`	Model Eloquent untuk entitas/tabel GalleryEntry.
`app/Models/Order.php`	Model Eloquent untuk entitas/tabel Order.
`app/Models/OrderItem.php`	Model Eloquent untuk entitas/tabel OrderItem.
`app/Models/Payment.php`	Model Eloquent untuk entitas/tabel Payment.
`app/Models/Product.php`	Model Eloquent untuk entitas/tabel Product.
`app/Models/Service.php`	Model Eloquent untuk entitas/tabel Service.
`app/Models/SiteSetting.php`	Model Eloquent untuk entitas/tabel SiteSetting.
`app/Models/User.php`	Model Eloquent untuk entitas/tabel User.
`app/Notifications/AdminActivityNotification.php`	Format notifikasi admin database.
`app/Providers/AppServiceProvider.php`	Service provider dan data global view.
`app/Services/AdminNotifier.php`	Service aturan bisnis: AdminNotifier.
`app/Services/BookingAvailabilityService.php`	Service aturan bisnis: BookingAvailabilityService.
`app/Services/BookingTransactionService.php`	Service aturan bisnis: BookingTransactionService.
`app/Services/MidtransGateway.php`	Service aturan bisnis: MidtransGateway.
`app/Services/PaymentService.php`	Service aturan bisnis: PaymentService.
`app/Services/QueueNumberService.php`	Service aturan bisnis: QueueNumberService.
`app/Support/AdminResources.php`	Konfigurasi/dukungan CRUD admin.
`app/Support/PaymentPolicy.php`	Aturan batas waktu payment dan perhitungan DP.
`app/Support/ServiceSchedule.php`	Mengambil durasi global, jam buka/tutup, dan waktu mulai terakhir.
`artisan`	Entry point CLI Laravel untuk migrate, test, schedule, cache, dan command lain.
`bootstrap/app.php`	Membuat aplikasi, mendaftarkan routes, alias middleware admin, dan exception JSON.
`bootstrap/cache/.gitignore`	Cache bootstrap generated; jangan diedit manual.
`bootstrap/cache/packages.php`	Cache bootstrap generated; jangan diedit manual.
`bootstrap/cache/services.php`	Cache bootstrap generated; jangan diedit manual.
`bootstrap/providers.php`	Daftar service provider aplikasi.
`composer.json`	Daftar dependency PHP, autoload PSR-4, dan script Composer.
`composer.lock`	Versi dependency PHP yang terkunci/reproduksibel.
`config/app.php`	Konfigurasi Laravel untuk app; nilai dapat berasal dari .env.
`config/auth.php`	Konfigurasi Laravel untuk auth; nilai dapat berasal dari .env.
`config/barbershop.php`	Konfigurasi Laravel untuk barbershop; nilai dapat berasal dari .env.
`config/cache.php`	Konfigurasi Laravel untuk cache; nilai dapat berasal dari .env.
`config/database.php`	Konfigurasi Laravel untuk database; nilai dapat berasal dari .env.
`config/filesystems.php`	Konfigurasi Laravel untuk filesystems; nilai dapat berasal dari .env.
`config/logging.php`	Konfigurasi Laravel untuk logging; nilai dapat berasal dari .env.

File	Peran
`config/mail.php`	Konfigurasi Laravel untuk mail; nilai dapat berasal dari .env.
`config/payments.php`	Konfigurasi Laravel untuk payments; nilai dapat berasal dari .env.
`config/queue.php`	Konfigurasi Laravel untuk queue; nilai dapat berasal dari .env.
`config/services.php`	Konfigurasi Laravel untuk services; nilai dapat berasal dari .env.
`config/session.php`	Konfigurasi Laravel untuk session; nilai dapat berasal dari .env.
`database/.gitignore`	Berkas pendukung proyek.
`database/database.sqlite`	Berkas pendukung proyek.
`database/factories/UserFactory.php`	Factory data user untuk pengujian.
`database/migrations/0001_01_01_000000_create_users_table.php`	Migration: create users table.
`database/migrations/0001_01_01_000001_create_cache_table.php`	Migration: create cache table.
`database/migrations/0001_01_01_000002_create_jobs_table.php`	Migration: create jobs table.
`database/migrations/2026_08_23_000000_create_barbershop_tables.php`	Migration: create barbershop tables.
`database/migrations/2026_08_23_000100_create_catalog_tables.php`	Migration: create catalog tables.
`database/migrations/2026_09_04_000000_add_admin_and_workflow_fields.php`	Migration: add admin and workflow fields.
`database/migrations/2026_09_04_010000_add_pos_and_image_fields.php`	Migration: add pos and image fields.
`database/migrations/2026_09_05_000000_merge_completed_bookings_into_transactions.php`	Migration: merge completed bookings into transactions.
`database/migrations/2026_09_07_000000_add_erp_booking_fields.php`	Migration: add erp booking fields.
`database/migrations/2026_09_07_000100_create_payments_table.php`	Migration: create payments table.
`database/migrations/2026_09_07_000200_create_notifications_table.php`	Migration: create notifications table.
`database/migrations/2026_09_07_000300_add_work_hours_to_barbers.php`	Migration: add work hours to barbers.
`database/migrations/2026_09_07_000400_rebrand_to_homcuts.php`	Migration: rebrand to homcuts.
`database/migrations/2026_09_07_000500_replace_gallery_with_homcuts_photos.php`	Migration: replace gallery with homcuts photos.
`database/migrations/2026_09_07_000600_enforce_unique_service_names.php`	Migration: enforce unique service names.
`database/migrations/2026_09_07_000700_move_gallery_asset_paths.php`	Migration: move gallery asset paths.
`database/migrations/2026_09_09_000800_replace_product_catalog.php`	Migration: replace product catalog.
`database/migrations/2026_09_12_000900_add_daily_queue_numbers.php`	Migration: add daily queue numbers.
`database/migrations/2026_09_12_001000_unify_service_queue_and_schedule.php`	Migration: unify service queue and schedule.
`database/migrations/2026_09_15_001100_rename_visible_barber_terms_to_capster.php`	Migration: rename visible barber terms to capster.
`database/migrations/2026_09_22_001300_standardize_service_duration_and_order_status.php`	Menyeragamkan data durasi layanan dan status order.
`database/migrations/2026_09_22_001400_add_global_service_duration_setting.php`	Menambahkan pengaturan durasi layanan global.
`database/migrations/2026_09_22_001500_add_store_operating_hours_settings.php`	Menambahkan pengaturan jam buka dan tutup.
`database/migrations/2026_09_22_001600_add_payment_expiry_setting.php`	Menambahkan pengaturan batas waktu pembayaran.
`database/seeders/AdminUserSeeder.php`	Membuat/menaikkan hak admin dari environment.

File	Peran
`database/seeder/BarbershopSeeder.php`	Mengisi capster, layanan, produk, galeri, dan setting awal.
`database/seeder/DatabaseSeeder.php`	Menjalankan seeder katalog dan admin.
`database/seeder/data/homcuts_gallery.php`	Dataset seed galeri HOMCUTS.
`database/seeder/data/homcuts_products.php`	Dataset seed produk HOMCUTS.
`lang/id/validation.php`	Terjemahan pesan validasi Bahasa Indonesia.
`package-lock.json`	Versi dependency npm yang terkunci.
`package.json`	Dependency dan script npm/Vite.
`phpunit.xml`	Konfigurasi PHPUnit dan environment default testing.
`public/.htaccess`	Rewrite Apache menuju index.php (berguna pada deployment Apache).
`public/barber-team-photo.png`	Aset gambar publik HOMCUTS.
`public/favicon.ico`	Ikon tab browser.
`public/gallery-images/homcuts-look-01.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-02.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-03.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-04.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-05.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-06.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-07.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-08.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-09.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-10.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-11.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-12.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-13.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-14.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-15.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-16.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-17.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-18.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-19.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-20.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-21.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-22.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-23.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-24.jpg`	Aset foto galeri HOMCUTS.
`public/gallery-images/homcuts-look-25.jpg`	Aset foto galeri HOMCUTS.
`public/haircut-gallery.png`	Aset gambar publik HOMCUTS.
`public/homcuts-logo-transparent.png`	Aset gambar publik HOMCUTS.
`public/homcuts-logo.jpg`	Aset gambar publik HOMCUTS.

File	Peran
`public/homcuts-storefront.png`	Aset gambar publik HOMCUTS.
`public/homcuts-team.jpg`	Aset gambar publik HOMCUTS.
`public/index.php`	Front controller web; semua request Laravel masuk melalui file ini.
`public/og.png`	Gambar Open Graph/fallback sosial.
`public/product-catalog-photo.png`	Aset gambar publik HOMCUTS.
`public/product-images/gatsby-fiber-clay-80g.png`	Aset foto produk katalog.
`public/product-images/gatsby-nature-styling-balm-clay-75g.png`	Aset foto produk katalog.
`public/product-images/gatsby-powder-clay-unbreakable-smooth-20g.png`	Aset foto produk katalog.
`public/product-images/gatsby-styling-pomade-supreme-hold-30g.png`	Aset foto produk katalog.
`public/product-images/gatsby-water-gloss-hyper-solid-150g.png`	Aset foto produk katalog.
`public/product-images/his-erha-hair-dust-12g.png`	Aset foto produk katalog.
`public/product-images/kahf-volumized-suave-hair-powder-10g.png`	Aset foto produk katalog.
`public/product-images/makarizo-advisor-hair-scalp-tonic-145ml.png`	Aset foto produk katalog.
`public/product-images/natur-hair-tonic-ginseng-90ml.png`	Aset foto produk katalog.
`public/product-images/smith-ocean-dust-11g.png`	Aset foto produk katalog.
`public/robots.txt`	Petunjuk crawler web.
`resources/css/app.css`	Tailwind theme, komponen CSS, marquee, dan responsive admin table.
`resources/js/app.js`	Interaksi pelanggan: menu, booking availability, filter, cart, checkout, polling status.
`resources/views/about.blade.php`	Blade halaman publik: about.
`resources/views/admin/account.blade.php`	Blade panel admin: account.
`resources/views/admin/auth/login.blade.php`	Blade halaman login admin.
`resources/views/admin/dashboard.blade.php`	Blade panel admin: dashboard.
`resources/views/admin/layouts/app.blade.php`	Layout Blade panel admin dan script live update.
`resources/views/admin/pos/create.blade.php`	Blade formulir Kasir POS.
`resources/views/admin/resources/form.blade.php`	Blade generik list/form resource admin.
`resources/views/admin/resources/index.blade.php`	Blade generik list/form resource admin.
`resources/views/booking.blade.php`	Blade halaman publik: booking.
`resources/views/contact.blade.php`	Blade halaman publik: contact.
`resources/views/gallery.blade.php`	Blade halaman publik: gallery.
`resources/views/home.blade.php`	Blade halaman publik: home.
`resources/views/layouts/app.blade.php`	Blade halaman publik: app.
`resources/views/partials/cart.blade.php`	Partial Blade keranjang/checkout yang dipakai layout publik.
`resources/views/payments/show.blade.php`	Blade status/struk pembayaran pelanggan.
`resources/views/shop.blade.php`	Blade halaman publik: shop.
`resources/views/welcome.blade.php`	Blade halaman publik: welcome.
`routes/console.php`	Command payments:expire dan scheduler setiap menit.
`routes/web.php`	Seluruh route HTTP publik dan admin.
`storage/app/.gitignore`	Placeholder/berkas runtime storage; bukan logika aplikasi.
`storage/app/private/.gitignore`	Placeholder/berkas runtime storage; bukan logika aplikasi.

File	Peran
`storage/app/private/bookings.jsonl`	Placeholder/berkas runtime storage; bukan logika aplikasi.
`storage/app/private/orders.jsonl`	Placeholder/berkas runtime storage; bukan logika aplikasi.
`storage/app/public/.gitignore`	Placeholder/berkas runtime storage; bukan logika aplikasi.
`tests/Feature/AdminTest.php`	Feature test: Admin.
`tests/Feature/BarbershopTest.php`	Feature test: Barbershop.
`tests/Feature/BlackBoxTest.php`	Feature test: BlackBox.
`tests/Feature/ErpWorkflowTest.php`	Feature test: ErpWorkflow.
`tests/Feature/ExampleTest.php`	Feature test: Example.
`tests/Feature/PosTest.php`	Feature test: Pos.
`tests/TestCase.php`	Base class test Laravel.
`tests/Unit/ExampleTest.php`	Unit test dasar.
`vite.config.js`	Input CSS/JS, plugin Laravel/Tailwind, dan watch Vite.

B.3 Folder yang sengaja tidak dirinci per file

Folder	Mengapa tidak diedit langsung
vendor/	Dependency Composer; puluhan ribu file pihak ketiga, dibuat dari composer.lock.
node_modules/	Dependency npm; dibuat dari package-lock.json.
public/build/	Output Vite ber-hash; dibuat ulang oleh npm run build.
bootstrap/cache/	Cache discovery/config framework.
storage/framework/	Session, cache, dan compiled view runtime.
storage/logs/	Log runtime aplikasi.
public/storage	Symlink ke storage/app/public untuk berkas upload.

BAB LAMPIRAN C

Indeks class dan fungsi

Indeks ini membantu mencari lokasi logika. Method private/protected dijelaskan sebagai detail internal, sedangkan public umumnya menjadi API kelas/controller.

File	Class	Method	Tujuan ringkas
app/Http/Controllers/Admin/AccountController.php	AccountController	public edit()	Menampilkan form perubahan.
app/Http/Controllers/Admin/AccountController.php	AccountController	public update()	Memvalidasi dan menyimpan perubahan.
app/Http/Controllers/Admin/AuthController.php	AuthController	public create()	Menampilkan form pembuatan.
app/Http/Controllers/Admin/AuthController.php	AuthController	public store()	Memvalidasi dan menyimpan data baru.
app/Http/Controllers/Admin/AuthController.php	AuthController	public destroy()	Logout atau menghapus record sesuai konteks.
app/Http/Controllers/Admin/DashboardController.php	DashboardController	public __invoke()	Method internal sesuai konteks kelas.
app/Http/Controllers/Admin/NotificationController.php	NotificationController	public index()	Menampilkan daftar/JSON data.
app/Http/Controllers/Admin/NotificationController.php	NotificationController	public read()	Method internal sesuai konteks kelas.
app/Http/Controllers/Admin/NotificationController.php	NotificationController	public readAll()	Method internal sesuai konteks kelas.
app/Http/Controllers/Admin/NotificationController.php	NotificationController	private requestedIds()	Membaca/menormalisasi parameter request.
app/Http/Controllers/Admin/PaymentController.php	PaymentController	public __construct()	Method internal sesuai konteks kelas.
app/Http/Controllers/Admin/PaymentController.php	PaymentController	public confirmCash()	Mengonfirmasi pembayaran tunai.
app/Http/Controllers/Admin/PosController.php	PosController	public __construct()	Method internal sesuai konteks kelas.
app/Http/Controllers/Admin/PosController.php	PosController	public create()	Menampilkan form pembuatan.
app/Http/Controllers/Admin/PosController.php	PosController	public store()	Memvalidasi dan menyimpan data baru.
app/Http/Controllers/Admin/PosController.php	PosController	public availability()	Memeriksa ketersediaan slot.
app/Http/Controllers/Admin/PosController.php	PosController	private ensureServiceTimeNotPast()	Validasi aturan khusus dan menolak jika tidak terpenuhi.
app/Http/Controllers/Admin/ResourceController.php	ResourceController	public __construct()	Method internal sesuai konteks kelas.
app/Http/Controllers/Admin/ResourceController.php	ResourceController	public index()	Menampilkan daftar/JSON data.
app/Http/Controllers/Admin/ResourceController.php	ResourceController	public create()	Menampilkan form pembuatan.
app/Http/Controllers/Admin/ResourceController.php	ResourceController	public store()	Memvalidasi dan menyimpan data baru.
app/Http/Controllers/Admin/ResourceController.php	ResourceController	public edit()	Menampilkan form perubahan.
app/Http/Controllers/Admin/ResourceController.php	ResourceController	public update()	Memvalidasi dan menyimpan perubahan.
app/Http/Controllers/Admin/ResourceController.php	ResourceController	public destroy()	Logout atau menghapus record sesuai konteks.

File	Class	Method	Tujuan ringkas
app/Http/Controllers/Admin/ResourceController.php	ResourceController	public bookingAvailability()	Method internal sesuai konteks kelas.
app/Http/Controllers/Admin/ResourceController.php	ResourceController	private findRecord()	Method internal sesuai konteks kelas.
app/Http/Controllers/Admin/ResourceController.php	ResourceController	private validated()	Method internal sesuai konteks kelas.
app/Http/Controllers/Admin/ResourceController.php	ResourceController	private storeUploads()	Method internal sesuai konteks kelas.
app/Http/Controllers/Admin/ResourceController.php	ResourceController	private deleteManagedImage()	Menghapus berkas/record terkelola.
app/Http/Controllers/Admin/ResourceController.php	ResourceController	private deleteUploadedPath()	Menghapus berkas/record terkelola.
app/Http/Controllers/Admin/ResourceController.php	ResourceController	private saveBooking()	Method internal sesuai konteks kelas.
app/Http/Controllers/Admin/ResourceController.php	ResourceController	private applyTransactionFilters()	Menerapkan filter/sort/perubahan internal.
app/Http/Controllers/Admin/ResourceController.php	ResourceController	private sortOptions()	Method internal sesuai konteks kelas.
app/Http/Controllers/Admin/ResourceController.php	ResourceController	private applySort()	Menerapkan filter/sort/perubahan internal.
app/Http/Controllers/Admin/ResourceController.php	ResourceController	private resourceFilters()	Method internal sesuai konteks kelas.
app/Http/Controllers/Admin/ResourceController.php	ResourceController	private applyResourceFilters()	Menerapkan filter/sort/perubahan internal.
app/Http/Controllers/BookingController.php	BookingController	public __construct()	Method internal sesuai konteks kelas.
app/Http/Controllers/BookingController.php	BookingController	public store()	Memvalidasi dan menyimpan data baru.
app/Http/Controllers/BookingController.php	BookingController	public availability()	Memeriksa ketersediaan slot.
app/Http/Controllers/BookingController.php	BookingController	private ensureFutureTime()	Validasi aturan khusus dan menolak jika tidak terpenuhi.
app/Http/Controllers/ContactController.php	ContactController	public store()	Memvalidasi dan menyimpan data baru.
app/Http/Controllers/Controller.php	Controller	—	Tidak mendefinisikan method langsung.
app/Http/Controllers/MidtransWebhookController.php	MidtransWebhookController	public __construct()	Method internal sesuai konteks kelas.
app/Http/Controllers/MidtransWebhookController.php	MidtransWebhookController	public __invoke()	Method internal sesuai konteks kelas.
app/Http/Controllers/MidtransWebhookController.php	MidtransWebhookController	private amountMatches()	Method internal sesuai konteks kelas.
app/Http/Controllers/OrderController.php	OrderController	public __construct()	Method internal sesuai konteks kelas.
app/Http/Controllers/OrderController.php	OrderController	public store()	Memvalidasi dan menyimpan data baru.
app/Http/Controllers/PageController.php	PageController	public home()	Method internal sesuai konteks kelas.
app/Http/Controllers/PageController.php	PageController	public booking()	Method internal sesuai konteks kelas.
app/Http/Controllers/PageController.php	PageController	public shop()	Method internal sesuai konteks kelas.
app/Http/Controllers/PageController.php	PageController	public gallery()	Method internal sesuai konteks kelas.
app/Http/Controllers/PaymentController.php	PaymentController	public __construct()	Method internal sesuai konteks kelas.
app/Http/Controllers/PaymentController.php	PaymentController	public show()	Menampilkan detail/status.
app/Http/Controllers/PaymentController.php	PaymentController	public status()	Mengembalikan JSON status terbaru.
app/Http/Controllers/PaymentController.php	PaymentController	private expireIfNeeded()	Method internal sesuai konteks kelas.

File	Class	Method	Tujuan ringkas
app/Http/Middleware/EnsureUserIsAdmin.php	EnsureUserIsAdmin	public handle()	Memproses middleware dan meneruskan/menolak request.
app/Models/Barber.php	Barber	protected casts()	Menentukan konversi tipe atribut model.
app/Models/Barber.php	Barber	public galleryEntries()	Method internal sesuai konteks kelas.
app/Models/Barber.php	Barber	public bookings()	Method internal sesuai konteks kelas.
app/Models/Booking.php	Booking	protected casts()	Menentukan konversi tipe atribut model.
app/Models/Booking.php	Booking	public barber()	Method internal sesuai konteks kelas.
app/Models/Booking.php	Booking	public service()	Method internal sesuai konteks kelas.
app/Models/Booking.php	Booking	public orders()	Method internal sesuai konteks kelas.
app/Models/Booking.php	Booking	public transaction()	Method internal sesuai konteks kelas.
app/Models/ContactMessage.php	ContactMessage	—	Tidak mendefinisikan method langsung.
app/Models/GalleryEntry.php	GalleryEntry	protected casts()	Menentukan konversi tipe atribut model.
app/Models/GalleryEntry.php	GalleryEntry	public barber()	Method internal sesuai konteks kelas.
app/Models/Order.php	Order	public items()	Method internal sesuai konteks kelas.
app/Models/Order.php	Order	public booking()	Method internal sesuai konteks kelas.
app/Models/Order.php	Order	public cashier()	Method internal sesuai konteks kelas.
app/Models/Order.php	Order	public payments()	Method internal sesuai konteks kelas.
app/Models/Order.php	Order	public latestPayment()	Method internal sesuai konteks kelas.
app/Models/Order.php	Order	public getQueueCodeAttribute()	Accessor kode antrean Axxx.
app/Models/Order.php	Order	public getBarberNameAttribute()	Accessor nama capster dari item/booking.
app/Models/Order.php	Order	protected casts()	Menentukan konversi tipe atribut model.
app/Models/OrderItem.php	OrderItem	public order()	Method internal sesuai konteks kelas.
app/Models/OrderItem.php	OrderItem	public product()	Method internal sesuai konteks kelas.
app/Models/OrderItem.php	OrderItem	public service()	Method internal sesuai konteks kelas.
app/Models/OrderItem.php	OrderItem	public barber()	Method internal sesuai konteks kelas.
app/Models/Payment.php	Payment	protected casts()	Menentukan konversi tipe atribut model.
app/Models/Payment.php	Payment	public getRouteKeyName()	Memilih kolom route model binding.
app/Models/Payment.php	Payment	public order()	Method internal sesuai konteks kelas.
app/Models/Payment.php	Payment	public confirmer()	Method internal sesuai konteks kelas.
app/Models/Product.php	Product	protected casts()	Menentukan konversi tipe atribut model.
app/Models/Service.php	Service	protected casts()	Menentukan konversi tipe atribut model.
app/Models/Service.php	Service	protected details()	Accessor ringkasan layanan.
app/Models/Service.php	Service	public bookings()	Method internal sesuai konteks kelas.
app/Models/SiteSetting.php	SiteSetting	—	Tidak mendefinisikan method langsung.
app/Models/User.php	User	protected casts()	Menentukan konversi tipe atribut model.

File	Class	Method	Tujuan ringkas
app/Notifications/AdminActivityNotification.php	AdminActivityNotification	public __construct()	Method internal sesuai konteks kelas.
app/Notifications/AdminActivityNotification.php	AdminActivityNotification	public via()	Menentukan channel notifikasi.
app/Notifications/AdminActivityNotification.php	AdminActivityNotification	public toArray()	Membentuk payload notifikasi.
app/Providers/AppServiceProvider.php	AppServiceProvider	public register()	Mendaftarkan service/container binding.
app/Providers/AppServiceProvider.php	AppServiceProvider	public boot()	Menyiapkan data/aturan saat aplikasi boot.
app/Services/AdminNotifier.php	AdminNotifier	public send()	Mengirim notifikasi ke semua admin.
app/Services/BookingAvailabilityService.php	BookingAvailabilityService	public resolve()	Mencari slot/capster yang valid.
app/Services/BookingAvailabilityService.php	BookingAvailabilityService	private timeOnDate()	Method internal sesuai konteks kelas.
app/Services/BookingAvailabilityService.php	BookingAvailabilityService	public applyToBooking()	Menyalin hasil slot ke model Booking.
app/Services/BookingAvailabilityService.php	BookingAvailabilityService	private hasConflict()	Method internal sesuai konteks kelas.
app/Services/BookingTransactionService.php	BookingTransactionService	public __construct()	Method internal sesuai konteks kelas.
app/Services/BookingTransactionService.php	BookingTransactionService	public syncBooking()	Menyinkronkan booking menjadi order/item.
app/Services/BookingTransactionService.php	BookingTransactionService	public recordCompletedBooking()	Mencatat booking selesai tanpa duplikasi.
app/Services/MidtransGateway.php	MidtransGateway	public provision()	Method internal sesuai konteks kelas.
app/Services/MidtransGateway.php	MidtransGateway	public validSignature()	Method internal sesuai konteks kelas.
app/Services/PaymentService.php	PaymentService	public __construct()	Method internal sesuai konteks kelas.
app/Services/PaymentService.php	PaymentService	public createForOrder()	Membuat payment attempt untuk order.
app/Services/PaymentService.php	PaymentService	public markPaid()	Menerapkan transisi pembayaran lunas.
app/Services/PaymentService.php	PaymentService	public confirmCash()	Mengonfirmasi pembayaran tunai.
app/Services/PaymentService.php	PaymentService	public expireDuePayments()	Memproses payment/order yang melewati tenggat.
app/Services/PaymentService.php	PaymentService	public expire()	Mengedaluwarsakan satu payment.
app/Services/PaymentService.php	PaymentService	public cancelOrder()	Membatalkan order secara aman.
app/Services/PaymentService.php	PaymentService	public fail()	Menandai payment gagal.
app/Services/PaymentService.php	PaymentService	private cancelLockedOrder()	Method internal sesuai konteks kelas.
app/Services/PaymentService.php	PaymentService	private lockOrder()	Mengunci record untuk konsistensi transaksi.
app/Services/PaymentService.php	PaymentService	private lockOrderAndPayment()	Mengunci record untuk konsistensi transaksi.
app/Services/QueueNumberService.php	QueueNumberService	public assign()	Memberi nomor antrean global.
app/Support/AdminResources.php	AdminResources	—	Tidak mendefinisikan method langsung.

Cara memakai indeks

Mulai dari route/controller, lalu cari method public. Jika method itu memanggil service, gunakan baris service untuk mengikuti logika. Method private biasanya merupakan potongan validasi, lock, filter, upload, atau cleanup.

BAB LAMPIRAN D

Glosarium dan perintah cepat

Ringkasan istilah yang sering muncul saat membaca proyek.

Istilah	Arti pada proyek
Artisan	CLI Laravel (php artisan ...).
Blade	Template HTML server-side Laravel.
Controller	Koordinator request/response.
CSRF	Token pelindung form dari request lintas situs.
Eloquent	ORM Laravel untuk model, relasi, dan query.
Environment	Konfigurasi berjalan dari .env/variabel container.
Foreign key	Aturan referensi antartabel.
Idempotent	Diulang tetap memberi hasil akhir yang sama tanpa efek ganda.
Migration	Riwayat perubahan struktur database.
Middleware	Pemeriksaan sebelum/sesudah controller.
Model binding	Laravel mengubah parameter URL menjadi object model.
ORM	Pemetaan object PHP ke tabel SQL.
Polling	Meminta status berkala tanpa reload penuh.
Row lock	Penguncian baris selama transaksi untuk mencegah race.
Seeder	Pengisian data awal/default.
Session	Penyimpanan identitas login antar-request.
Snapshot transaksi	Salinan nama/harga/durasi saat order dibuat.
Throttle	Pembatasan jumlah request per waktu.
Vite	Build/dev server aset frontend.
Volume Docker	Penyimpanan data di luar lifecycle container.
Dependency injection	Laravel memasukkan object yang dibutuhkan ke constructor atau method, alih-alih class membuat semua dependency sendiri.
Service container	Object registry Laravel yang mengetahui cara membuat controller, service, dan dependency lain.
Flash message	Data session sementara, misalnya pesan sukses yang ditampilkan setelah redirect.
HTTP response	Object hasil yang membawa status, header, cookie, dan body kembali ke browser.

D.1 Cheat sheet

```
# Jalankan
docker compose up --build -d

# Status dan log
docker compose ps
docker compose logs -f web

# Informasi Laravel
docker compose exec web php artisan about
docker compose exec web php artisan route:list
docker compose exec web php artisan migrate:status

# Clear cache aman
docker compose exec web php artisan optimize:clear

# Test black-box / seluruh suite
docker compose --profile test run --rm test
docker compose --profile test run --rm test php vendor/bin/phpunit --do-not-cache-result

# Hentikan tanpa menghapus volume
docker compose down
```

D.2 Urutan belajar yang disarankan

1. Buka routes/web.php dan cocokkan URL dengan halaman.
2. Ikuti PageController untuk request baca sederhana.
3. Ikuti ContactController untuk POST sederhana.
4. Ikuti BookingController → BookingAvailabilityService → BookingTransactionService.
5. Ikuti OrderController dan PosController untuk transaksi/stok.
6. Baca PaymentService terakhir karena paling banyak aturan status/lock.
7. Baca test untuk menguji apakah pemahaman Anda sesuai perilaku nyata.

Penutup

Anda tidak harus menghafal semua API Laravel. Yang penting adalah dapat mengikuti arus data, mengetahui batas tanggung jawab tiap lapisan, dan selalu menguji perubahan terhadap database testing terpisah.